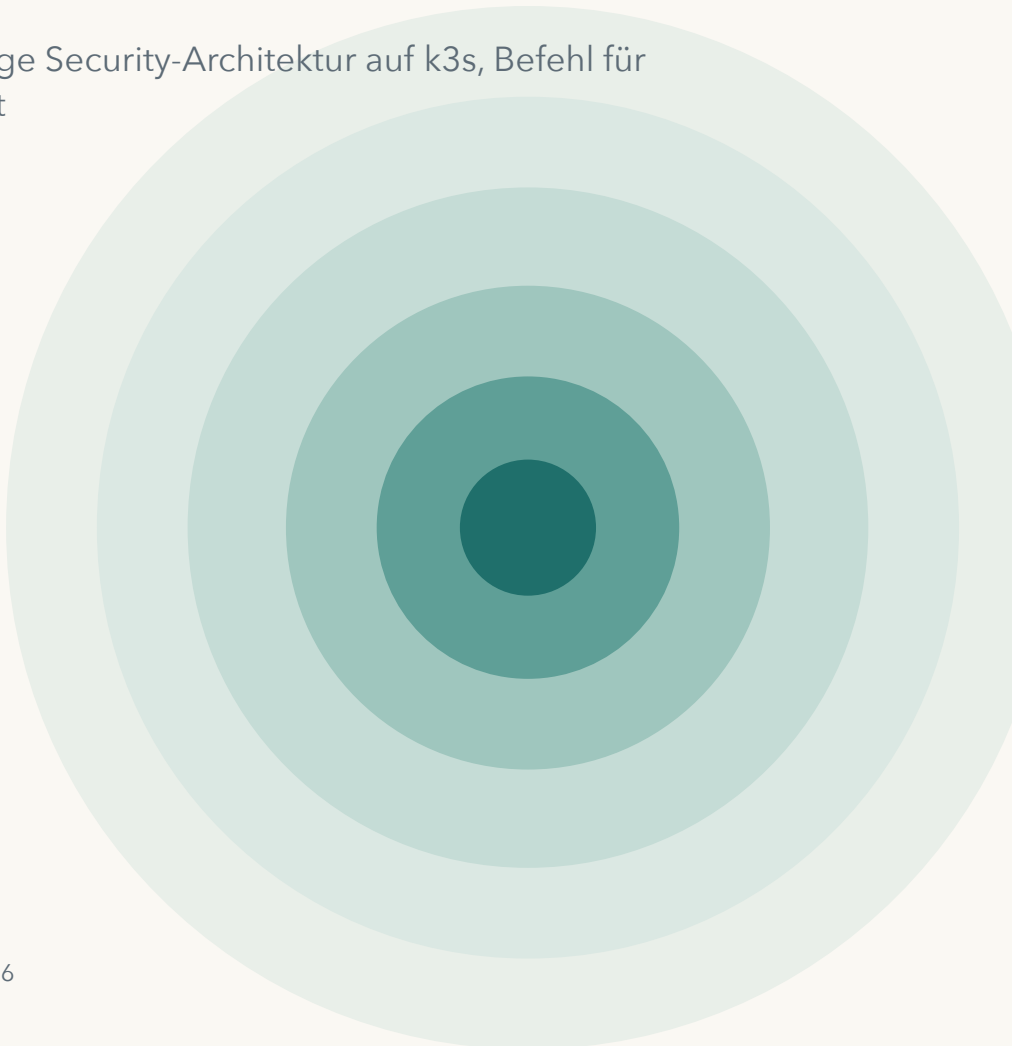
HANDS-ON-HANDBUCH

Kubernetes absichern

Eine durchgängige Security-Architektur auf k3s, Befehl für Befehl aufgebaut



Thomas Zachmann

Stand: 6. November 2026

Inhalt

So benutzt du dieses Buch 6
Zuerst das Repo klonen 6
Wo du einen Befehl ausführst 6
Die Kästen 7
Was geprüft wurde und was nicht 7
Was du brauchst 8
1 Die Architektur im Überblick 10
1.1 Das Grundprinzip: Verteidigung in Schichten 10
1.2 Die vier Ebenen deiner Architektur 10
1.3 Der Aufbau dieses Buches 11
1.4 Der Ressourcenplan 12
2 Das Fundament: k3s und Cilium 14
2.1 Was du brauchst 14
2.2 Die VM vorbereiten 14
2.3 k3s installieren 15
2.4 Die Kubeconfig in deine bestehende integrieren 17
2.5 Den neuen Kontext nutzen 18
2.6 Cilium installieren 19
2.7 Hubble ausprobieren 20
3 RBAC: Wer darf was? 22
3.1 Das Modell 22
3.2 Den Lab-Namespace anlegen 22
3.3 Rechte eines ServiceAccounts 23
3.4 Das Token gar nicht erst mounten 24
3.5 Ein echter Mensch: alice 24
3.6 Gefährliche Rechte erkennen 26
4 Pod Security Admission 28
4.1 Der Türsteher 28
4.2 Wer braucht welche Stufe? 28
4.3 Die Wirkung testen, bevor sie zuschlägt 29
4.4 Ausnahmen setzen 29
4.5 Den Clusterstandard festlegen 30

4.6 Den Standard testen 31
4.7 `demo` auf `restricted` setzen 32
5 Kyverno: Eigene Regeln 34
5.1 Wofür Kyverno, wenn es PSA gibt? 34
5.2 Installieren 34
5.3 Die Richtlinien schreiben 35
5.4 Lokal testen, ohne Cluster 36
5.5 Im Audit-Modus starten 38
5.6 Durchsetzen 38
5.7 Wenn Kyverno ausfällt 39
6 Default-Deny mit Cilium 42
6.1 Warum Default-Deny 42
6.2 Die Lab-Anwendung 42
6.3 Schritt 1: Alles sperren außer DNS 44
6.4 Schritt 2: Gezielt freigeben 45
6.5 Schritt 3: Regeln auf HTTP-Ebene 46
6.6 Schritt 4: Ausgang nach Hostname erlauben 47
6.7 Was weiterhin durchkommt 48
6.8 Wie du das auf echte Namespaces überträgst 48
7 Falco: Verdächtiges erkennen 50
7.1 Die Alarmanlage im Kernel 50
7.2 Installieren 50
7.3 Einen Angriff simulieren 51
7.4 Fehlalarme abstellen 53
8 Trivy Operator: Regelmäßig scannen 56
8.1 Bekannte Schwächen finden 56
8.2 Installieren 56
8.3 Ein altes Image finden lassen 57
8.4 Fehlkonfigurationen finden 58
8.5 Die Regel, die alles verbindet 59
9 Audit-Logs: Wer hat was getan? 60
9.1 Die Überwachungskamera des API-Servers 60
9.2 Die vier Protokoll-Stufen 60
9.3 Die Audit-Policy schreiben 60

9.4 Auf dem Node aktivieren ... 61
9.5 Das Log auswerten ... 62
9.6 Nützliche Auswertungen ... 63
10 Zentrale Sicht: Grafana und Loki ... 66
10.1 Was du aufbaust ... 66
10.2 SeaweedFS starten ... 66
10.3 Ein eigenes S3-Konto für Loki ... 67
10.4 Prometheus und Grafana installieren ... 68
10.5 Loki installieren ... 70
10.6 Alloy sammelt die Logs ein ... 72
10.7 Hubble und Trivy anbinden ... 74
10.8 Grafana öffnen ... 75
10.9 Audit-Logs abfragen ... 76
10.10 Falco-Meldungen abfragen ... 76
10.11 Trivy-Ergebnisse abfragen ... 76
11 Tempo: Dem Weg einer Anfrage folgen ... 78
11.1 Drei Arten von Signalen ... 78
11.2 Bucket und S3-Konto ... 78
11.3 Tempo installieren ... 79
11.4 Alloy nimmt Traces an ... 80
11.5 Der erste Trace ... 82
11.6 Der API-Server liefert Traces ... 82
11.7 Von Traces zu Logs springen ... 84
12 Alles zusammenführen ... 86
12.1 Das Sicherheits-Dashboard ... 86
12.2 Alarme ... 87
12.3 Die Angriffsübung ... 88
12.4 Die Spurensuche ... 90
12.5 Was die Übung gezeigt hat ... 90
13 Betrieb und Fehlersuche ... 92
13.1 Der Ablauf nach jedem Neustart ... 92
13.2 Was du sichern musst ... 92
13.3 Das Fehlersuch-Muster ... 93
13.4 Die Notbremsen ... 93

13.5 Updates ... 94
13.6 Ressourcen im Blick ... 94
Zusatzteil ... 96
14 Zusatz: cert-manager ... 98
14.1 Warum Automatisierung ... 98
14.2 Installieren ... 98
14.3 Die eigene CA aufbauen ... 99
14.4 Das erste Zertifikat ... 100
14.5 Erneuerung ausprobieren ... 101
14.6 Grafana per HTTPS erreichbar machen ... 101
15 Zusatz: OpenBao als Tresor ... 104
15.1 Warum ein Tresor ... 104
15.2 Das TLS-Zertifikat vorbereiten ... 104
15.3 Installieren ... 105
15.4 Initialisieren und entsiegeln ... 107
15.5 Erste Schritte im Tresor ... 107
15.6 Rechte vergeben: die Policy ... 108
15.7 Versiegeln erleben ... 109
16 Zusatz: External Secrets ... 112
16.1 Wie die Teile zusammenspielen ... 112
16.2 Installieren ... 112
16.3 Das Konto für den Zugriff ... 113
16.4 OpenBao vertraut Kubernetes ... 113
16.5 Die Verbindung beschreiben ... 114
16.6 Das erste externe Secret ... 115
16.7 Ändern und mitverfolgen ... 115
16.8 Wer darf jetzt was? ... 116
16.9 Lokis Zugangsdaten umziehen ... 117
Anhang ... 120
Versionen ... 120
Dein Repo am Ende ... 120
Spickzettel ... 121
LogQL-Spickzettel ... 121
Offizielle Dokumentation ... 122

So benutzt du dieses Buch

Einzelne Sicherheitstools sind schnell installiert. Schwer ist etwas anderes: zu verstehen, welches Tool welche Lücke schließt, wo sie sich überschneiden und was passiert, wenn eines ausfällt. Genau das lernst du hier, und zwar an deinem eigenen Cluster `k3s2`.

Du baust die Architektur Schicht für Schicht auf. Jedes Kapitel hat denselben Ablauf: Du erfährst zuerst, welches Problem gelöst wird. Dann tippst du Befehl für Befehl, prüfst das Ergebnis und probierst selbst etwas aus. Lange Skripte gibt es nicht.

Zuerst das Repo klonen

Alle Konfigurationsdateien, auf die sich das Buch bezieht, liegen in einem öffentlichen Repository auf GitHub: github.com/thomaszachmann/k3s. Hol es dir, bevor du mit Kapitel 2 beginnst.

MAC Schritt 1

```
git clone https://github.com/thomaszachmann/k3s.git
```

Lädt das Repo mit allen Dateien in den neuen Ordner `k3s`.

MAC Schritt 2

```
cd k3s
```

Wechselt ins Wurzelverzeichnis des Repos. Von hier aus laufen alle Befehle mit dem Etikett **Mac**.

Wo du einen Befehl ausführst

Jeder Befehl trägt ein farbiges Etikett. Es sagt dir, wo du dich gerade befindest:

Etikett	Bedeutung
MAC	Dein Terminal auf dem Laptop, im Wurzelverzeichnis dieses Repos.
NODE	Eine SSH-Sitzung auf der VM `k3s2` (172.16.0.136).
POD	Eine Shell in einem Container, die du vorher mit `kubectl exec` geöffnet hast.

Ersetze `<user>` jeweils durch deinen SSH-Login. Alle Pfade wie `security/psa.yaml` sind relativ zum Repo.

Die Kästen

WARUM DAS WICHTIG IST
Hier steht der Hintergrund, der dir beim nächsten Problem hilft.

STOLPERFALLE
Hier steht, was typischerweise schiefgeht und woran du es erkennst.

DAS SOLLTEST DU SEHEN
Hier steht das erwartete Ergebnis. Weicht deine Ausgabe ab, geh nicht weiter, sondern kläre zuerst den Unterschied.

PROBIER ES SELBST
Hier bist du dran. Überlege zuerst selbst, bevor du nachschlägst.

Was geprüft wurde und was nicht

Ein Buch voller Befehle ist nur so gut wie seine Prüfung. Deshalb steht hier ehrlich, wie weit sie ging.

Geprüft am 6. November 2026:

- Alle Chart-Versionen stammen aus den offiziellen Helm-Repositories.
- Alle Helm-Werte in diesem Buch wurden mit `helm template` gegen genau diese Chart-Versionen gerendert.
- Die drei Kyverno-Policies aus Kapitel 5 liefen mit der Kyverno-CLI 1.19.1 gegen gute und schlechte Beispiel-Pods.
- `ValidatingPolicy`, `ClusterSecretStore` und `ExternalSecret` wurden gegen die CRD-Schemas der Charts geprüft.
- Die Image-Tags der Lab-Workloads wurden in der Registry nachgeschlagen.

Nicht geprüft:

- Das Buch wurde **nicht** komplett auf einem echten Cluster durchgespielt. Der Testcluster beim Schreiben lief noch mit Flannel und Kubernetes 1.37.
- Die Alloy-Konfiguration, die Falco-Regeln, die `CiliumNetworkPolicy`-Manifeste, die TraceQL-Abfragen und die `bao`-Befehle folgen der offiziellen Doku, liefen aber nicht gegen die echten Programme.

Rechne also damit, dass an einer Stelle etwas abweicht. Das ist kein Grund zur Sorge, sondern Übung: Die Fehlersuche gehört zum Lernen, und Kapitel 13 hilft dir dabei.

Was du brauchst

- Eine frische Ubuntu-24.04-VM mit 4 CPUs, 8 GB RAM und mindestens 40 GB Disk. Außer Ubuntu ist nichts installiert.
- Auf dem Laptop `kubectl`, `helm`, `jq`, `openssl`, `curl` und die Kyverno-CLI.
- Das geklonte Repo mit `k3s/config.yaml` und `cilium/values.yaml`, dazu `git` auf dem Laptop.
- Rund zwei Wochenenden Zeit, oder ein Kapitel pro Abend.

EIN WORT ZUR REIHENFOLGE

Das Buch folgt deinem Plan: erst RBAC und PSA, dann Kyverno und Network Policies, dann Falco, dann Trivy und zum Schluss Audit-Logs mit Grafana, Loki und Tempo. Kapitel 1 erklärt, warum das auch ohne frühes Monitoring gut funktioniert. cert-manager, OpenBao und External Secrets stehen als freiwilliger Zusatzteil am Ende.

KAPITEL 1

Die Architektur im Überblick

Du verstehst, welche Schicht welchen Angriff abfängt und warum der Aufbau genau so abläuft.

Dauer: 20 Minuten Lesen

1.1 Das Grundprinzip: Verteidigung in Schichten

Keine einzelne Maßnahme ist perfekt. Ein Angreifer findet vielleicht eine Lücke in deiner Web-Anwendung, und schon liegt Code in deinem Cluster. Die Frage ist dann nicht mehr, ob etwas passiert ist, sondern wie weit er kommt und ob du es bemerkst.

Deshalb baust du mehrere unabhängige Schichten. Jede geht davon aus, dass die davor versagt hat. Die Reise eines Angriffs zeigt, wie das zusammenspielt:

Phase	Was der Angreifer versucht	Wer greift ein
Vorbereiten	Ein verwundbares Image läuft im Cluster	Trivy, Kyverno
Eindringen	Er nutzt die Lücke und landet im Pod	Falco meldet es
Ausbreiten	Er ruft andere Dienste oder das Internet auf	Cilium blockiert
Rechte ausweiten	Er nutzt das Service-Account-Token	RBAC begrenzt
Ausbrechen	Er startet einen privilegierten Pod	PSA, Kyverno
Beute holen	Er sucht Passwörter	RBAC, Audit-Log
Spuren verwischen	Er löscht lokale Logs	Loki hält sie außerhalb

1.2 Die vier Ebenen deiner Architektur

Es hilft, die Tools nach dem Zeitpunkt zu ordnen, an dem sie wirken:

Ebene	Frage	Tools
Vor dem Start	Darf das überhaupt ins Cluster?	RBAC, PSA, Kyverno
Zur Laufzeit	Was darf es tun, wenn es läuft?	Cilium, Falco
Dauerhaft	Wo stecken bekannte Schwächen?	Trivy
Nachweisen	Was ist tatsächlich passiert?	Audit-Logs, Grafana, Loki, Tempo

Damit siehst du auch, wo sich Tools überschneiden und wo nicht:

- **PSA und Kyverno** prüfen beide beim Anlegen. PSA kennt nur drei feste Stufen, Kyverno versteht beliebige eigene Regeln. PSA ist die einfache Grundlage, Kyverno die Ergänzung.
- **Falco und Cilium** wirken beide zur Laufzeit. Falco beobachtet und meldet, Cilium blockiert. Falco kann also nichts verhindern, aber es sieht Dinge, die im Netzwerk nie auftauchen.
- **Trivy und Kyverno** ergänzen sich. Trivy findet Schwächen, Kyverno kann verhindern, dass bestimmte Images überhaupt starten.
- **Loki und Tempo** speichern Verschiedenes: Loki die Log-Zeilen, Tempo den Weg einer einzelnen Anfrage durch das System.

1.3 Der Aufbau dieses Buches

Du gehst von innen nach außen vor: erst die Rechte, dann die Regeln, dann das Netz, dann das Erkennen, und am Ende führst du alles zentral zusammen.

Kap.	Baustein	Rolle
2	k3s und Cilium auf der Ubuntu-VM, Kubeconfig	Fundament
3	RBAC	Zugriff
4	Pod Security Admission	Zulassung
5	Kyverno	Zulassung
6	Cilium Default-Deny	Laufzeit
7	Falco	Laufzeit
8	Trivy Operator	Dauerhaft
9	Kubernetes-Audit-Logs	Nachweis
10	Grafana, Prometheus, Loki	Zentrale Sicht
11	Tempo	Zentrale Sicht
12	Dashboard und Angriffsübung	Zusammenführung
13	Betrieb und Fehlersuche	Dauerbetrieb

Die Kapitel 14 bis 16 sind ein freiwilliger Zusatzteil zu TLS und Secrets mit cert-manager, OpenBao und External Secrets. Für den Hauptteil brauchst du sie nicht.

Funktioniert das ohne Monitoring am Anfang?

Ja. Beim Sperren musst du sehen, was gerade blockiert wird, und das kannst du von Anfang an:

Kapitel	Deine Sicht, solange Grafana noch fehlt
RBAC	`kubectl auth can-i`

Kapitel	Deine Sicht, solange Grafana noch fehlt
PSA	Warnungen von `kubectl`, `--dry-run=server`
Kyverno	`kubectl get polr -A`
Cilium	`hubble observe --verdict DROPPED`
Falco	`kubectl logs`
Trivy	`kubectl get vuln -A`
Audit-Logs	`jq` direkt auf dem Node

Eines musst du dafür vorausplanen: Monitoring-Dienste brauchen Host-Zugriff, der Namespace `monitoring` bekommt seine PSA-Ausnahme daher schon in Kapitel 4, lange bevor dort etwas läuft.

Die Regel in jedem Kapitel

Unabhängig vom Tool gilt immer dieselbe Abfolge:

1. **Melden.** Regeln laufen im Audit-Modus und blockieren nichts.
2. **Auswerten.** Du liest die Meldungen und passt Regeln an.
3. **Durchsetzen.** Erst jetzt wird wirklich blockiert.

1.4 Der Ressourcenplan

Deine VM hat 8 GB RAM. Die Tabelle zeigt grobe Schätzungen für den Speicherbedarf, keine Messwerte. Vergleiche sie mit `kubectl top pods -A`, sobald alles läuft.

Komponente	RAM (geschätzt)
k3s selbst, CoreDNS, Traefik, Metrics Server	1,0 GB
Cilium, Hubble	0,5 GB
Kyverno	0,4 GB
Falco	0,3 GB
Trivy Operator (Scans kurzzeitig mehr)	0,3 GB
Prometheus, Grafana, Alertmanager, Exporter	1,5 GB
SeaweedFS, Loki, Alloy	0,8 GB
Tempo	0,3 GB
Summe Hauptteil	**etwa 5,4 GB**

Es bleibt Luft für deine Lab-Anwendung, aber nicht viel. Der Zusatzteil braucht rund 0,3 GB mehr. Trivy lässt du nur einen Scan gleichzeitig ausführen, das ist in Kapitel 8 bereits eingestellt.

PROBIER ES SELBST

Nimm dir dein eigenes Szenario: Ein Angreifer hat eine Remote-Shell im `frontend`-Pod. Schreibe drei Dinge auf, die er als Nächstes versuchen würde, und notiere, welche Schicht jeweils reagieren sollte. In Kapitel 12 spielst du genau das durch und vergleichst.

WAS DU GELERNT HAST

- Jede Schicht geht davon aus, dass die vorherige versagt hat.
- Tools lassen sich danach ordnen, wann sie wirken: vor dem Start, zur Laufzeit, dauerhaft und im Nachhinein.
- Sichtbarkeit beginnt nicht erst mit Grafana, `kubectl` und Hubble reichen für den Anfang.
- Erst melden, dann auswerten und zuletzt durchsetzen.

KAPITEL 2

Das Fundament: k3s und Cilium

Aus einer leeren Ubuntu-VM wird ein Cluster mit Cilium, und dein Laptop steuert ihn über einen eigenen Kontext.

Dauer: 60 Minuten **Voraussetzung:** Eine Ubuntu-VM mit SSH-Zugang

2.1 Was du brauchst

Dieses Kapitel startet bei null. Auf der VM läuft nichts außer Ubuntu.

Punkt	Dieses Buch nimmt an
Betriebssystem	Ubuntu 24.04
Größe	4 CPUs, 8 GB RAM, mindestens 40 GB Disk
Name und Adresse	`k3s2` mit fester IP `172.16.0.136`
Laptop	`kubectl`, `helm`, `jq`, `openssl`, `curl`, und dieses Repo

Die IP-Adresse steht fest in `cilium/values.yaml`. Hat deine VM eine andere, ändere sie dort, und ersetze sie in allen Befehlen dieses Buches. Sie darf sich später nicht mehr ändern, gib der VM also eine feste IP oder eine DHCP-Reservierung.

LÄUFT AUF DER VM SCHON K3S?

Dann entferne es zuerst mit `sudo /usr/local/bin/k3s-uninstall.sh`. Das löscht den Cluster samt Daten unwiderruflich. Das Netzwerk-Plugin legt man bei k3s am besten bei der Installation fest, ein Umbau im laufenden Cluster lohnt sich nicht.

2.2 Die VM vorbereiten

MAC Schritt 1

```
scp k3s/config.yaml <user>@172.16.0.136:/tmp/config.yaml
```

Kopiert deine k3s-Konfiguration auf die VM. Sie enthält die Flags, die Flannel und kube-proxy abschalten. Gleich schauen wir sie an.

MAC Schritt 2

```
ssh <user>@172.16.0.136
```

Öffnet die Sitzung auf der VM. Alle folgenden Schritte mit dem Etikett **Node** laufen hier.

NODE Schritt 3

```
sudo hostnamectl set-hostname k3s2
```

Der Hostname wird zum Node-Namen im Cluster. Lege ihn deshalb vor der Installation fest.

NODE Schritt 4

```
sudo apt update && sudo apt -y upgrade
```

Bringt das System auf den aktuellen Stand. Das ist die erste Sicherheitsmaßnahme überhaupt.

NODE Schritt 5

```
sudo apt install -y curl jq
```

`curl` holt gleich das Installationsskript, `jq` brauchst du später für die Audit-Logs.

NODE Schritt 6

```
uname -r
```

Zeigt die Kernel-Version. Cilium verlangt mindestens 5.10, Ubuntu 24.04 liefert 6.8.

NODE Schritt 7

```
sudo ufw status
```

Ubuntus Firewall ist meist inaktiv. Bei `inactive` machst du einfach weiter.

Ist die Firewall aktiv, müssen API-Server, Pods und Services durchkommen:

NODE Schritt 8

```
sudo ufw allow 6443/tcp
sudo ufw allow from 10.42.0.0/16 to any
sudo ufw allow from 10.43.0.0/16 to any
```

Das sind die Regeln aus der k3s-Doku. `10.42.0.0/16` ist das Pod-Netz, `10.43.0.0/16` das Service-Netz. Die k3s-Doku empfiehlt alternativ, `ufw` ganz abzuschalten. Lass Port 22 offen, sonst sperrst du dich aus.

2.3 k3s installieren

NODE Schritt 9

```
sudo install -D -m 600 -o root -g root \
  /tmp/config.yaml /etc/rancher/k3s/config.yaml
```

`install` kopiert die Datei und setzt Rechte in einem Schritt. `-D` legt fehlende Verzeichnisse an, `-m 600` macht die Datei nur für root lesbar. Sie muss **vor** der Installation hier liegen, sonst startet k3s mit Flannel.

NODE Schritt 10

```
rm /tmp/config.yaml
sudo grep -v '^#' /etc/rancher/k3s/config.yaml | grep .
```

Entfernt die Kopie und zeigt die aktiven Zeilen. Jede davon hat einen Zweck:

Einstellung	Zweck
`flannel-backend: "none"`	Kein Standard-CNI, Cilium übernimmt
`disable-network-policy: true`	Kein zweiter Policy-Controller neben Cilium
`disable-kube-proxy: true`	Cilium ersetzt auch kube-proxy
`cluster-cidr`, `service-cidr`	Die Netze, muss zu Cilium passen
`cluster-init: true`	etcd statt SQLite, erlaubt Snapshots
`secrets-encryption: true`	Secrets liegen verschlüsselt im Datastore

NODE Schritt 11

```
curl -sfL https://get.k3s.io | INSTALL_K3S_CHANNEL=v1.36 sh -
```

Installiert k3s aus dem Kanal `v1.36`. Das ist wichtig: Der Kanal `stable` liefert 1.37, Cilium 1.20 ist aber nur bis 1.36 getestet.

NODE Schritt 12

```
sudo k3s kubectl get nodes
```

Fragt den Cluster direkt auf der VM ab.

DAS SOLLTEST DU SEHEN

Der Node `k3s2` steht auf `NotReady`, Version `v1.36.x+k3s1`. Das ist richtig, denn es gibt noch kein CNI. Falls k3s nicht startet, zeigt `sudo journalctl -u k3s -n 50` den Grund.

NODE Schritt 13

```
sudo k3s secrets-encrypt status
```

Bestätigt, dass Secrets im Datastore verschlüsselt werden.

NODE Schritt 14

```
sudo install -m 600 -o "$USER" \
  /etc/rancher/k3s/k3s.yaml /tmp/k3s2.yaml
```

Die Kubeconfig gehört root. Das legt eine Kopie an, die dein Nutzer per `scp` holen darf. Nur du kannst sie lesen, denn sie enthält volle Cluster-Rechte.

NODE Schritt 15

```
exit
```

Zurück zum Laptop.

2.4 Die Kubeconfig in deine bestehende integrieren

In `~/.kube/config` liegen schon andere Kontexte. Du darfst die Datei also nicht überschreiben, sondern fügst den neuen Cluster hinzu.

MAC Schritt 16

```
cp -p ~/.kube/config ~/.kube/config.bak-k3s2
```

Sicherung. Falls etwas schiefgeht, kopierst du diese Datei zurück.

MAC Schritt 17

```
scp <user>@172.16.0.136:/tmp/k3s2.yaml ~/.kube/k3s2.raw
ssh <user>@172.16.0.136 'rm /tmp/k3s2.yaml'
```

Holt die Datei auf den Laptop und löscht die Admin-Kopie auf der VM.

MAC Schritt 18

```
sed -e 's/127\.0\.0\.1/172.16.0.136/' \
    -e 's/: default$/: k3s2/' \
    ~/.kube/k3s2.raw > ~/.kube/k3s2.yaml
```

Die Datei zeigt auf `127.0.0.1` und nennt alles `default`. `sed` ersetzt die Adresse und benennt Cluster, Nutzer und Kontext in `k3s2` um. So kollidiert nichts mit deinen anderen Kontexten. Das `$` trifft nur Zeilenenden, die Zertifikatsdaten bleiben unberührt.

MAC Schritt 19

```
chmod 600 ~/.kube/k3s2.yaml
```

Nur du darfst diese Datei lesen.

MAC Schritt 20

```
kubectl config get-contexts
```

Zeigt deine Kontexte. Steht `k3s2` dort schon aus einer früheren Installation, musst du den alten Eintrag entfernen:

MAC Schritt 21

```
kubectl config delete-context k3s2
kubectl config delete-cluster k3s2
kubectl config delete-user k3s2
```

Nur nötig, wenn `k3s2` schon existiert. Alte Einträge würden beim Zusammenführen gewinnen, und `kubectl` meldete dann `x509`-Fehler.

MAC Schritt 22

```
KUBECONFIG=~/.kube/config:~/.kube/k3s2.yaml \
  kubectl config view --flatten > ~/.kube/config.neu
```

`KUBECONFIG` mit zwei Dateien lässt `kubectl` beide zusammenführen. `--flatten` schreibt die Zertifikate direkt in die Ausgabe, damit die Datei allein funktioniert.

MAC Schritt 23

```
chmod 600 ~/.kube/config.neu
mv ~/.kube/config.neu ~/.kube/config
```

Setzt die neue Datei mit den richtigen Rechten ein.

MAC Schritt 24

```
rm ~/.kube/k3s2.raw ~/.kube/k3s2.yaml
```

Räumt die Zwischendateien mit den Admin-Zugangsdaten auf.

2.5 Den neuen Kontext nutzen

MAC Schritt 25

```
kubectl config use-context k3s2
```

Ab jetzt gehen alle `kubectl`- und `helm`-Befehle an diesen Cluster.

MAC Schritt 26

```
kubectl config get-contexts
kubectl get nodes
```

Der Stern markiert den aktiven Kontext. Deine anderen Kontexte sind noch da.

DAS SOLLTEST DU SEHEN

Der Stern steht bei `k3s2`, und du siehst vom Laptop aus denselben `NotReady`-Node. Prüfe den Kontext künftig immer, bevor du etwas löschst.

2.6 Cilium installieren

Dein `cilium/values.yaml` enthält schon alles Nötige. Die beiden wichtigsten Zeilen sind `k8sServiceHost` und `k8sServicePort`.

WARUM DAS WICHTIG IST

Ohne kube-proxy gibt es zunächst niemanden, der die Service-IP `10.43.0.1` übersetzt, das erledigt ja Cilium selbst. Der Cilium-Agent müsste den API-Server aber erreichen, um zu starten. Dieses Henne-Ei-Problem löst die direkte Adresse `172.16.0.136:6443`.

MAC — Schritt 27

```
helm upgrade --install cilium \
  oci://quay.io/cilium/charts/cilium \
  --version 1.20.4 -n kube-system \
  -f cilium/values.yaml --wait
```

`upgrade --install` funktioniert beim ersten Mal und bei jeder Wiederholung. `--wait` kehrt erst zurück, wenn alle Pods bereit sind.

MAC — Schritt 28

```
kubectl get nodes
```

Jetzt sollte der Node auf `Ready` stehen.

MAC — Schritt 29

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  cilium-dbg status --brief
```

Fragt den Agenten direkt. `ds/cilium` wählt einen Pod des DaemonSets, `-c` den Container.

MAC — Schritt 30

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  cilium-dbg status | grep KubeProxyReplacement
```

Bestätigt, dass Cilium den kube-proxy ersetzt. Erwartet ist `True`.

MAC — Schritt 31

```
kubectl run dns-test --rm -it --restart=Never \
  --image=busybox:1.36 -- \
  nslookup kubernetes.default.svc.cluster.local
```

Startet einen Wegwerf-Pod und testet in einem Schritt Pod-Netz, DNS und Service-Routing. `--rm` entfernt ihn danach.

DAS SOLLTEST DU SEHEN

`nslookup` antwortet mit der Adresse `10.43.0.1`. Das beweist: CoreDNS ist erreichbar, und die Service-IP funktioniert ohne kube-proxy.

2.7 Hubble ausprobieren

Hubble ist Ciliums Beobachtungswerkzeug. Es zeigt jede Netzwerkverbindung im Cluster mit der Entscheidung, ob sie erlaubt wurde. Bis zum Monitoring in Kapitel 10 ist es dein wichtigstes Auge.

MAC Schritt 32

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  hubble observe --last 10
```

Zeigt die letzten zehn Netzwerkflüsse auf dem Node. Der Hubble-Client steckt im Agent-Image, du musst nichts installieren.

MAC Schritt 33

```
kubectl -n kube-system port-forward svc/hubble-ui 12000:80
```

Macht die grafische Oberfläche unter http://localhost:12000 erreichbar. Beenden mit `Ctrl-C`.

PROBIER ES SELBST

Öffne in einem zweiten Terminal `hubble observe --follow` (mit dem `exec`-Präfix von oben) und starte erneut den `dns-test`. Findest du die DNS-Anfrage zu `kube-dns` im Strom der Flüsse?

WAS DU GELERNT HAST

- Das CNI legst du bei der Installation fest, nicht danach.
- Die `config.yaml` muss vor der Installation auf der VM liegen.
- Ohne kube-proxy muss Cilium den API-Server direkt kennen.
- `KUBECONFIG` mit `--flatten` führt Kubeconfigs zusammen, ohne andere Kontexte zu gefährden.
- Mit Hubble siehst du ab jetzt jede Verbindung im Cluster.

KAPITEL 3

RBAC: Wer darf was?

Du vergibst minimale Rechte an Menschen und an Workloads und prüfst sie mit `kubectl auth can-i`.

Dauer: 60 Minuten **Voraussetzung:** Kapitel 2

3.1 Das Modell

RBAC (Role-Based Access Control) besteht aus vier Objekten:

Objekt	Bedeutung
`Role`	Rechte innerhalb eines Namespace
`ClusterRole`	Rechte clusterweit oder als wiederverwendbare Vorlage
`RoleBinding`	Vergibt eine Rolle in einem Namespace
`ClusterRoleBinding`	Vergibt eine Rolle im ganzen Cluster

Der Grundsatz lautet: Alles ist verboten, was nicht ausdrücklich erlaubt ist. Rollen können nur Rechte hinzufügen, nie wegnehmen.

Rechte bekommen drei Arten von Subjekten: **User**, **Group** und **ServiceAccount**. Kubernetes kennt keine Benutzerdatenbank. User und Gruppen ergeben sich aus dem Zertifikat oder Token, mit dem sich jemand anmeldet. ServiceAccounts dagegen sind echte Objekte im Cluster.

3.2 Den Lab-Namespace anlegen

MAC Schritt 1

```
kubectl create namespace demo
```

In diesem Namespace übst du für den Rest des Buches.

MAC Schritt 2

```
kubectl label namespace demo security.k3s2/tier=app
```

Dieses Label nutzt Kyverno in Kapitel 5, um Anwendungs-Namespaces zu erkennen.

3.3 Rechte eines ServiceAccounts

Jeder Pod läuft unter einem ServiceAccount. Ohne Angabe ist das `default`.

MAC Schritt 3

```
kubectl auth can-i --list \
  --as=system:serviceaccount:demo:default -n demo
```

`--as` lässt dich als dieses Konto fragen, ohne dich anzumelden. Das nennt sich Impersonation. Du siehst fast nur Rechte zum Selbst-Erkennen.

MAC Schritt 4

```
kubectl create serviceaccount app -n demo
```

Ein eigenes Konto für deine Anwendung. So teilt sie ihre Rechte mit niemandem.

MAC Schritt 5

```
kubectl create role pod-leser -n demo \
  --verb=get,list,watch --resource=pods
```

Erlaubt nur Lesen von Pods in `demo`.

MAC Schritt 6

```
kubectl create rolebinding app-liest-pods -n demo \
  --role=pod-leser --serviceaccount=demo:app
```

Vergibt die Rolle an das Konto `app`. Erst das Binding macht die Rolle wirksam.

MAC Schritt 7

```
kubectl auth can-i list pods -n demo \
  --as=system:serviceaccount:demo:app
```

Erwartet: `yes`.

MAC Schritt 8

```
kubectl auth can-i delete pods -n demo \
  --as=system:serviceaccount:demo:app
```

Erwartet: `no`. Lesen war erlaubt, Löschen nicht.

MAC Schritt 9

```
kubectl auth can-i list pods -n kube-system \
  --as=system:serviceaccount:demo:app
```

Erwartet: `no`. Die Rolle gilt nur in `demo`.

3.4 Das Token gar nicht erst mounten

Kubernetes legt das Token des ServiceAccounts automatisch in jeden Pod. Die meisten Anwendungen sprechen aber nie mit dem API-Server. Für sie ist das Token nur ein Geschenk an den Angreifer.

DATEI `security/rbac/sa-ohne-token.yaml`

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: frontend
  namespace: demo
automountServiceAccountToken: false
```

Pods dieses Kontos bekommen kein Token. Du kannst das pro Pod überschreiben, wenn einer es wirklich braucht.

MAC Schritt 10

```
kubectl apply -f security/rbac/sa-ohne-token.yaml
```

Dieses Konto nutzt das `frontend` in Kapitel 6.

3.5 Ein echter Mensch: alice

Jetzt gibst du einem Menschen Zugang, und zwar über ein Client-Zertifikat. `CN` wird dabei zum Benutzernamen, `O` zur Gruppe.

MAC Schritt 11

```
mkdir -p ~/k3s2-alice && cd ~/k3s2-alice
```

Ein eigener Ordner außerhalb des Repos, denn hier liegt gleich ein privater Schlüssel.

MAC Schritt 12

```
openssl genrsa -out alice.key 2048
```

Erzeugt alices privaten Schlüssel. Er verlässt ihren Rechner nie.

MAC Schritt 13

```
openssl req -new -key alice.key -out alice.csr \
  -subj "/CN=alice/O=plattform"
```

Erstellt einen Zertifikatsantrag für Benutzer `alice` in Gruppe `plattform`.

Lege den Antrag als Kubernetes-Objekt an:

DATEI ~/k3s2-alice/csr.yaml

```yaml
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: alice
spec:
  request: HIER_BASE64
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400
  usages:
    - client auth
```

`expirationSeconds: 86400` begrenzt das Zertifikat auf einen Tag. Das ist wichtig, weil Kubernetes Client-Zertifikate **nicht widerrufen** kann.

MAC Schritt 14

```
base64 < alice.csr | tr -d '\n'
```

Gibt den Antrag als eine Base64-Zeile aus. Setze sie in `csr.yaml` anstelle von `HIER_BASE64` ein.

MAC Schritt 15

```
kubectl apply -f csr.yaml
kubectl certificate approve alice
```

Reicht den Antrag ein und genehmigt ihn. Jetzt signiert der Cluster.

MAC Schritt 16

```
kubectl get csr alice \
  -o jsonpath='{.status.certificate}' | base64 -d > alice.crt
```

Holt das fertige Zertifikat.

MAC Schritt 17

```
kubectl config view --raw --minify \
  -o jsonpath='{.clusters[0].cluster.certificate-authority-data}' \
  | base64 -d > k3s2-ca.crt
```

Speichert das CA-Zertifikat des Clusters. Damit prüft alice, dass sie wirklich mit deinem API-Server spricht. Es ist öffentlich und nicht geheim.

MAC Schritt 18

```
kubectl config set-cluster k3s2 --kubeconfig=alice.kubeconfig \
  --server=https://172.16.0.136:6443 \
  --certificate-authority=k3s2-ca.crt --embed-certs
```

Trägt den Cluster in eine neue Kubeconfig ein. Deine eigene bleibt unberührt.

MAC Schritt 19

```
kubectl config set-credentials alice \
  --kubeconfig=alice.kubeconfig \
  --client-certificate=alice.crt --client-key=alice.key \
  --embed-certs
```

Trägt alices Identität ein.

MAC Schritt 20

```
kubectl config set-context alice --kubeconfig=alice.kubeconfig \
  --cluster=k3s2 --user=alice --namespace=demo
kubectl config use-context alice --kubeconfig=alice.kubeconfig
```

Verbindet Cluster und Nutzer zu einem Kontext.

MAC Schritt 21

```
kubectl --kubeconfig=alice.kubeconfig get pods
```

Erwartet: `Forbidden`. Das ist richtig, denn alice ist angemeldet, hat aber noch keine Rechte. Authentifizierung (wer bist du?) und Autorisierung (was darfst du?) sind zwei Schritte.

MAC Schritt 22

```
kubectl create rolebinding plattform-edit -n demo \
  --clusterrole=edit --group=plattform
```

Vergibt die eingebaute Rolle `edit` an die **Gruppe**. So bekommt jede Person dieser Gruppe die Rechte, und du musst Bindings nicht pro Person pflegen.

MAC Schritt 23

```
kubectl --kubeconfig=alice.kubeconfig get pods
kubectl --kubeconfig=alice.kubeconfig get nodes
```

Pods gehen jetzt. Nodes sind weiter verboten, denn sie sind clusterweit, und das Binding gilt nur in `demo`.

MAC Schritt 24

```
cd -
```

Zurück ins Repo.

3.6 Gefährliche Rechte erkennen

Manche Rechte sehen harmlos aus, kommen aber einem Vollzugriff gleich:

Recht	Warum es gefährlich ist
`get secrets`	Liest alle Tokens und Passwörter des Namespace

Recht	Warum es gefährlich ist
`create pods`	Startet Pods mit jedem ServiceAccount des Namespace
`pods/exec`	Öffnet eine Shell in fremden Containern
`escalate`, `bind`	Vergibt Rechte, die man selbst nicht besitzt
`impersonate`	Handelt als beliebiger anderer Nutzer

MAC

Schritt 25

```
kubectl get clusterrolebindings -o json | jq -r '
  .items[]
  | select(.roleRef.name == "cluster-admin")
  | .metadata.name'
```

Listet alle Bindings mit vollen Admin-Rechten. Das ist deine wichtigste Prüfliste: Was hier steht, hat die Kontrolle über alles.

PROBIER ES SELBST

Warum ist `create pods` so gefährlich? Stell dir vor, im Namespace liegt ein ServiceAccount mit hohen Rechten. Wie käme ein Nutzer, der nur Pods erstellen darf, an dessen Rechte? Der Abschnitt zum Token im Pod hilft dir dabei.

WAS DU GELERNT HAST

- Rollen fügen nur Rechte hinzu, alles andere ist verboten.
- Authentifizierung und Autorisierung sind getrennte Schritte.
- `kubectl auth can-i --as` prüft Rechte, ohne sich anzumelden.
- Bindings an Gruppen sind besser als Bindings an Personen.
- Client-Zertifikate lassen sich nicht widerrufen, also halte sie kurzlebig.

KAPITEL 4

Pod Security Admission

Du legst fest, welche Pods überhaupt starten dürfen: clusterweit als Standard und pro Namespace strenger oder lockerer.

Dauer: 45 Minuten **Voraussetzung:** Kapitel 3

4.1 Der Türsteher

Pod Security Admission (PSA) prüft jeden neuen Pod, bevor er angelegt wird. Es ist fest in Kubernetes eingebaut. Du wählst pro Namespace eine Stufe und einen Modus.

Stufe	Bedeutung
`privileged`	Keine Einschränkungen. Für Systemkomponenten.
`baseline`	Verbietet bekannte Gefahren wie `privileged`, `hostNetwork`, `hostPath`.
`restricted`	Am strengsten: kein Root, Capabilities entfernt, Seccomp nötig.

Modus	Wirkung bei einem Verstoß
`enforce`	Der Pod wird abgelehnt.
`audit`	Der Pod startet, der Verstoß landet im Audit-Log.
`warn`	Der Pod startet, `kubectl` zeigt eine Warnung.

WARUM DAS WICHTIG IST

PSA prüft nur **beim Anlegen**. Laufende Pods bleiben unberührt, auch wenn du die Stufe verschärfst. Ein Problem zeigt sich dann erst beim nächsten Neustart. Prüfe also vorher, wen eine Änderung treffen würde.

4.2 Wer braucht welche Stufe?

Falco muss den Kernel beobachten und braucht einen privilegierten Container mit vielen `hostPath` -Mounts. `restricted` würde ihn sofort ablehnen. Dasselbe gilt für alles, was den Node selbst betrachtet.

Namespace	Stufe	Grund
`kube-system`	ausgenommen	Cilium, Traefik, CoreDNS brauchen Host-Zugriff
`monitoring`	`privileged`	node-exporter nutzt `hostNetwork` und `hostPID`, Alloy nutzt `hostPath`
`falco`	`privileged`	eBPF-Treiber, viele `hostPath`-Mounts
`trivy-system`	`privileged`	Der Node-Collector prüft Host-Dateien
`demo`	`restricted`	Deine Anwendungen
alle anderen	`baseline`	Clusterweiter Standard

Der Standard gilt auch für neue Namespaces. Wer eine Ausnahme braucht, muss sie sich aktiv holen. Das ist das Prinzip **secure by default**.

4.3 Die Wirkung testen, bevor sie zuschlägt

MAC — Schritt 1

```
kubectl label --dry-run=server --overwrite ns --all \
  pod-security.kubernetes.io/enforce=baseline
```

`--dry-run=server` lässt den API-Server die Änderung prüfen, ohne sie zu speichern. Er warnt dich vor jedem laufenden Pod, der gegen `baseline` verstößt.

DAS SOLLTEST DU SEHEN

Du bekommst Warnungen für `kube-system`, zum Beispiel wegen `hostPath` oder `hostNetwork`. Dieser Namespace braucht seine Ausnahme. Noch laufen keine weiteren Host-nahen Dienste, die kommen erst später.

4.4 Ausnahmen setzen

Setze die Ausnahmen **zuerst**, auch für Dienste, die erst später kommen. Sonst lehnt der neue Standard sie ab, sobald sie starten wollen.

MAC — Schritt 2

```
kubectl create namespace monitoring
kubectl create namespace falco
kubectl create namespace trivy-system
```

Legt die drei Namespaces schon jetzt an, damit sie ihre Stufe haben, bevor die ersten Pods darin starten.

MAC Schritt 3

```
kubectl label namespace monitoring \
  pod-security.kubernetes.io/enforce=privileged
```

Erlaubt in `monitoring` alles. Label am Namespace schlagen den clusterweiten Standard.

MAC Schritt 4

```
kubectl label namespace monitoring \
  pod-security.kubernetes.io/warn=baseline \
  pod-security.kubernetes.io/audit=baseline
```

Blockiert nichts, macht aber Verstöße sichtbar. So bleibt die Ausnahme nachvollziehbar.

MAC Schritt 5

```
kubectl label namespace falco trivy-system \
  pod-security.kubernetes.io/enforce=privileged
```

Gibt beiden die nötige Stufe.

4.5 Den Clusterstandard festlegen

DATEI `security/psa.yaml`

```yaml
apiVersion: apiserver.config.k8s.io/v1
kind: AdmissionConfiguration
plugins:
  - name: PodSecurity
    configuration:
      apiVersion: pod-security.admission.config.k8s.io/v1
      kind: PodSecurityConfiguration
      defaults:
        enforce: baseline
        enforce-version: latest
        audit: restricted
        audit-version: latest
        warn: restricted
        warn-version: latest
      exemptions:
        usernames: []
        runtimeClasses: []
        namespaces:
          - kube-system
```

Der Standard setzt `baseline` durch und meldet alles, was `restricted` nicht erfüllen würde. Nur `kube-system` ist ganz ausgenommen. Ausnahmen gelten für ganze Namespaces, halte die Liste also kurz.

MAC Schritt 6

```
scp security/psa.yaml <user>@172.16.0.136:/tmp/psa.yaml
```

Bringt die Datei auf den Node.

Kommentiere in `k3s/config.yaml` jetzt diese beiden Zeilen ein. Die `audit-`-Zeilen bleiben auskommentiert, sie sind für Kapitel 9.

DATEI `k3s/config.yaml (Ausschnitt)`

```
kube-apiserver-arg:
  - "admission-control-config-file=/var/lib/rancher/k3s/server/psa.yaml"
```

MAC Schritt 7

```
scp k3s/config.yaml <user>@172.16.0.136:/tmp/config.yaml
ssh <user>@172.16.0.136
```

Überträgt die Konfiguration und öffnet die Sitzung.

NODE Schritt 8

```
sudo install -m 600 -o root -g root /tmp/psa.yaml \
  /var/lib/rancher/k3s/server/psa.yaml
```

Die Datei muss vor dem Neustart bereitliegen.

NODE Schritt 9

```
sudo install -m 600 -o root -g root /tmp/config.yaml \
  /etc/rancher/k3s/config.yaml
rm /tmp/psa.yaml /tmp/config.yaml
sudo systemctl restart k3s
```

Setzt die Konfiguration ein und startet k3s neu.

NODE Schritt 10

```
exit
```

Zurück zum Mac.

4.6 Den Standard testen

MAC Schritt 11

```
kubectl create namespace psa-test
```

Ein frischer Namespace ohne Label, hier gilt also der Clusterstandard.

DATEI `security/psa-boese.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: boese
  namespace: psa-test
spec:
  hostNetwork: true
  containers:
    - name: c
      image: busybox:1.36
      command: [sleep, "3600"]
```

```
      securityContext:
        privileged: true
```

MAC Schritt 12

```
kubectl apply -f security/psa-boese.yaml
```

Erwartet: `Forbidden`. Die Meldung nennt beide Verstöße, `hostNetwork` und `privileged`.

MAC Schritt 13

```
kubectl delete namespace psa-test
```

Räumt auf.

4.7 `demo` auf `restricted` setzen

MAC Schritt 14

```
kubectl label namespace demo \
  pod-security.kubernetes.io/enforce=restricted
```

Ab jetzt laufen in `demo` nur noch Pods, die alle Regeln der strengsten Stufe einhalten.

MAC Schritt 15

```
kubectl run test --image=nginx:1.29-alpine -n demo
```

Erwartet: `Forbidden`. Lies die Meldung genau, denn sie sagt dir, welche Felder fehlen.

Feld im Pod	Zweck
`allowPrivilegeEscalation: false`	Prozesse können keine Rechte hinzugewinnen
`capabilities.drop: [ALL]`	Entfernt alle Linux-Sonderrechte
`runAsNonRoot: true`	Der Container startet nicht als root
`seccompProfile.type: RuntimeDefault`	Sperrt seltene, gefährliche Systemaufrufe

Normales `nginx` startet als root und läuft deshalb hier nicht. Du brauchst ein Image, das ohne root auskommt:

DATEI `security/psa-gut.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: gut
  namespace: demo
  labels:
    team: plattform
spec:
  securityContext:
```

```
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: web
      image: nginxinc/nginx-unprivileged:1.29-alpine
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop: [ALL]
      resources:
        limits:
          memory: 64Mi
```

MAC

Schritt 16

```
kubectl apply -f security/psa-gut.yaml
kubectl get pod gut -n demo
```

Dieser Pod erfüllt alle Regeln und läuft.

MAC

Schritt 17

```
kubectl delete pod gut -n demo
```

Räumt auf.

PROBIER ES SELBST

Starte in einem neuen Namespace einen normalen `nginx`-Pod. Er besteht `baseline`, verstößt aber gegen `restricted`. Welche Warnung zeigt `kubectl`? Der Modus `audit` schreibt denselben Verstoß ins Audit-Log, das du in Kapitel 9 aktivierst.

WAS DU GELERNT HAST

- PSA prüft nur beim Anlegen, laufende Pods bleiben unberührt.
- `--dry-run=server` zeigt vorab, wen eine Verschärfung trifft.
- Ausnahmen zuerst setzen, dann den Standard verschärfen.
- Der Standard `baseline` macht auch neue Namespaces automatisch sicherer.
- `restricted` braucht vier Einstellungen und ein Image, das ohne root läuft.

KAPITEL 5

Kyverno: Eigene Regeln

Du schreibst Richtlinien, die PSA nicht kennt, testest sie lokal und führst sie stufenweise ein.

Dauer: 60 Minuten **Voraussetzung:** Kapitel 4

5.1 Wofür Kyverno, wenn es PSA gibt?

PSA kennt nur drei feste Stufen und sagt nichts dazu, **was** in deinem Cluster gelten soll. Kyverno ergänzt genau das, zum Beispiel:

- Images brauchen einen festen Tag, kein `latest`.
- Jeder Container braucht ein Memory-Limit.
- Jeder Pod trägt das Label `team`.

Kyverno ist ein Admission-Webhook: Der API-Server fragt ihn vor dem Speichern um Erlaubnis. Eine Regel ist eine `ValidatingPolicy` mit einem CEL-Ausdruck. CEL ist die Ausdruckssprache, die auch Kubernetes selbst verwendet.

STOLPERFALLE

In vielen Anleitungen findest du noch `kind: ClusterPolicy`. Das ist seit Kyverno 1.19 veraltet und wird in 1.20 entfernt. Nutze `ValidatingPolicy` mit `apiVersion: policies.kyverno.io/v1`.

5.2 Installieren

DATEI `kyverno/values.yaml`

```yaml
admissionController:
  replicas: 1
backgroundController:
  replicas: 1
cleanupController:
  replicas: 1
reportsController:
  replicas: 1
```

Ein Node, also je eine Instanz. Das spart RAM, heißt aber auch: Fällt der Pod aus, ist Kyverno kurz nicht erreichbar.

MAC Schritt 1

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update
```

Trägt das Repository ein.

MAC Schritt 2

```
helm upgrade --install kyverno kyverno/kyverno \
  -n kyverno --create-namespace --version 3.9.2 \
  -f kyverno/values.yaml --wait --timeout 5m
```

Chart 3.9.2 enthält Kyverno 1.19.2. Der neue Namespace bekommt automatisch die Stufe `baseline`.

MAC Schritt 3

```
kubectl get pods -n kyverno
```

Vier Controller sollten laufen.

5.3 Die Richtlinien schreiben

DATEI `security/kyverno/policies.yaml`

```yaml
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: disallow-latest-tag
spec:
  validationActions: [Audit]
  matchConstraints:
    namespaceSelector:
      matchExpressions:
        - key: kubernetes.io/metadata.name
          operator: NotIn
          values: [kube-system, kyverno]
    resourceRules:
      - apiGroups: ['']
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Images brauchen einen festen Tag, nicht latest.
      expression: >-
        object.spec.containers.all(c,
          c.image.contains(':') && !c.image.endsWith(':latest'))
---
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: require-memory-limit
spec:
  validationActions: [Audit]
  matchConstraints:
    namespaceSelector:
      matchLabels:
        security.k3s2/tier: app
    resourceRules:
```

```
      - apiGroups: ['']
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Jeder Container braucht ein Memory-Limit.
      expression: >-
        object.spec.containers.all(c,
          c.?resources.?limits.?memory.orValue('') != '')
---
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: require-team-label
spec:
  validationActions: [Audit]
  matchConstraints:
    namespaceSelector:
      matchLabels:
        security.k3s2/tier: app
    resourceRules:
      - apiGroups: ['']
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Das Label team fehlt.
      expression: >-
        object.metadata.?labels.team.orValue('') != ''
```

So liest du die Regel:

- `validationActions: [Audit]` meldet nur. `Deny` würde blockieren.
- `matchConstraints` legt fest, wen die Regel betrifft. Die erste Regel gilt fast überall, die anderen nur in Namespaces mit dem Label `security.k3s2/tier=app`.
- `expression` ist der CEL-Ausdruck. Er muss `true` ergeben, damit der Pod besteht.
- `.all(c, ...)` prüft jeden Container, und `?.` greift sicher auf Felder zu, die fehlen dürfen. `orValue('')` liefert dann einen Ersatzwert.

STOLPERFALLE

Die Regel `disallow-latest-tag` prüft auf einen Doppelpunkt. Ein Image wie `registry:5000/app` hat zwar einen Doppelpunkt, aber keinen Tag, und würde durchrutschen. Für ein Lab reicht das, in Produktion würdest du gleich auf Digests prüfen.

5.4 Lokal testen, ohne Cluster

Die Kyverno-CLI führt Policies ohne Cluster aus. So findest du Fehler, bevor sie jemanden stören.

DATEI `security/kyverno/test/gut.yaml`

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: gut
  namespace: demo
  labels:
    team: plattform
spec:
  containers:
    - name: web
      image: nginxinc/nginx-unprivileged:1.29-alpine
      resources:
        limits:
          memory: 64Mi
```

DATEI `security/kyverno/test/schlecht.yaml`

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: schlecht
  namespace: demo
spec:
  containers:
    - name: web
      image: nginx:latest
```

DATEI `security/kyverno/test/values.yaml`

```yaml
apiVersion: cli.kyverno.io/v1alpha1
kind: Values
namespaceSelector:
  - name: demo
    labels:
      kubernetes.io/metadata.name: demo
      security.k3s2/tier: app
```

Offline kennt die CLI keine Namespaces. Diese Datei sagt ihr, welche Labels `demo` hat.

MAC Schritt 4

```
kyverno apply security/kyverno/policies.yaml \
  --resource security/kyverno/test/gut.yaml \
  --values-file security/kyverno/test/values.yaml
```

Prüft den guten Pod. Du hast die CLI bereits installiert (`brew install kyverno`).

DAS SOLLTEST DU SEHEN

Das Ergebnis lautet `pass: 3, fail: 0`. Beim Beispiel `schlecht.yaml` lautet es `pass: 0, fail: 3`, jeweils mit deiner Fehlermeldung. Genau das kam bei der Prüfung dieses Buches heraus.

MAC Schritt 5

```
kyverno apply security/kyverno/policies.yaml \
  --resource security/kyverno/test/schlecht.yaml \
  --values-file security/kyverno/test/values.yaml
```

Prüft den schlechten Pod. Alle drei Regeln sollten anschlagen.

5.5 Im Audit-Modus starten

MAC Schritt 6

```
kubectl apply -f security/kyverno/policies.yaml
```

Lädt die drei Regeln. Sie blockieren noch nichts.

MAC Schritt 7

```
kubectl get vpol
```

`vpol` ist die Kurzform für `validatingpolicies`. Alle drei sollten bereit sein.

MAC Schritt 8

```
kubectl get polr -A
```

`polr` steht für `policyreports`. Kyverno prüft auch bestehende Pods im Hintergrund und schreibt das Ergebnis hierher. Die Spalten `PASS` und `FAIL` zeigen den Stand pro Ressource.

MAC Schritt 9

```
kubectl get polr -A -o json | jq -r '
  .items[].results[]?
  | select(.result == "fail")
  | "\(.policy): \(.message)"' | sort | uniq -c
```

Fasst alle Verstöße zusammen. Das ist deine Arbeitsliste, bevor du irgendetwas durchsetzt.

5.6 Durchsetzen

Sieht der Bericht gut aus, stellst du um. Im Moment läuft in `demo` nichts, du kannst also gefahrlos beginnen.

Ändere in `security/kyverno/policies.yaml` bei `require-memory-limit` und `require-team-label` die Zeile auf `validationActions: [Deny]`. Die Regel zum `latest`-Tag bleibt vorerst auf `Audit`, weil sie fast den ganzen Cluster betrifft.

MAC Schritt 10

```
kubectl apply -f security/kyverno/policies.yaml
```

Aktiviert die Durchsetzung.

MAC Schritt 11

```
kubectl run verboten -n demo \
  --image=nginxinc/nginx-unprivileged:1.29-alpine
```

Erwartet wird eine Ablehnung, zuerst vermutlich von PSA, denn hier fehlen auch die `restricted`-Felder. Das zeigt schön, dass beide Schichten unabhängig prüfen.

MAC Schritt 12

```
kubectl apply --dry-run=server -f security/kyverno/test/gut.yaml
```

Prüft den Pod aus dem Test, ohne ihn anzulegen. PSA lehnt ihn ab, denn die Datei erfüllt `restricted` nicht. Kyverno allein hätte ihn durchgelassen.

STOLPERFALLE

Die Regeln greifen bei Pods. Ein Deployment mit Verstoß wird oft erst einmal angenommen, und nur seine Pods scheitern. Das Deployment zeigt dann `0/1`, und die Ursache steht in den Events: `kubectl get events -n demo`. Ob Kyverno auch das Deployment direkt ablehnt, regelt `spec.autogen`, siehe Doku.

5.7 Wenn Kyverno ausfällt

Ein Webhook kann den Cluster behindern: Ist er eingestellt, Anfragen abzulehnen, und nicht erreichbar, startet kein Pod mehr. Merke dir daher den Notausgang:

MAC Schritt 13

```
kubectl get validatingwebhookconfigurations
```

Listet alle Webhooks. Im Notfall löschst du den betroffenen Eintrag mit `kubectl delete`. Kyverno legt ihn beim nächsten Start wieder an.

PROBIER ES SELBST

Schreibe eine vierte Regel `disallow-default-sa`: Pods in App-Namespaces dürfen nicht den Service-Account `default` verwenden. Tipp: Das Feld heißt `object.spec.serviceAccountName`, und es darf auch ganz fehlen. Teste sie zuerst mit der CLI und ergänze deine Test-Pods.

WAS DU GELERNT HAST

- PSA liefert die Grundlage, Kyverno die Regeln deines Clusters.
- `ValidatingPolicy` ersetzt die veraltete `ClusterPolicy`.
- Mit der CLI testest du Policies lokal, ohne Cluster.
- Policy Reports zeigen dir vorab, was `Deny` treffen würde.
- Admission-Webhooks sind kritisch für den Cluster, kenne deinen Notausgang.

KAPITEL 6

Default-Deny mit Cilium

Pods dürfen nur noch das, was du ausdrücklich erlaubst, und Hubble zeigt dir jede blockierte Verbindung.

Dauer: 75 Minuten **Voraussetzung:** Kapitel 5

6.1 Warum Default-Deny

Standardmäßig darf in Kubernetes jeder Pod mit jedem anderen reden und ins Internet gehen. Für einen Angreifer ist das ideal: Ein Pod reicht, und er erreicht von dort alles. Bei **Default-Deny** drehst du das um: Nichts geht, außer dem, was du erlaubst.

Cilium arbeitet mit Identitäten statt IP-Adressen. Eine Regel sagt „Pods mit Label `app=frontend`“, nicht „10.42.0.7“. Das passt zu Kubernetes, wo sich IP-Adressen ständig ändern.

WARUM DAS WICHTIG IST

Sobald eine Policy einen Pod auswählt, gilt für die betroffene Richtung automatisch Default-Deny. Eine `ingress`-Regel sperrt also allen anderen eingehenden Verkehr. Das ist mächtig, aber auch die häufigste Ursache für Überraschungen.

6.2 Die Lab-Anwendung

Deine Anwendung hat zwei Teile: Ein `backend` (Webserver) und ein `frontend`, das es aufruft. Beide erfüllen `restricted` und deine Kyverno-Regeln.

DATEI `security/demo/app.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
```

```
        team: plattform
    spec:
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: web
          image: nginxinc/nginx-unprivileged:1.29-alpine
          ports:
            - containerPort: 8080
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: [ALL]
          resources:
            limits:
              memory: 64Mi
---
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: demo
spec:
  selector:
    app: backend
  ports:
    - port: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
        team: plattform
    spec:
      serviceAccountName: frontend
      securityContext:
        runAsNonRoot: true
        runAsUser: 100
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: curl
          image: curlimages/curl:8.17.0
          command: [sleep, "86400"]
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: [ALL]
          resources:
            limits:
              memory: 32Mi
```

MAC Schritt 1

```
kubectl apply -f security/demo/app.yaml
kubectl get pods -n demo
```

Beide Pods sollten auf `Running` stehen. Falls nicht, zeigt `kubectl get events -n demo` den Grund.

MAC Schritt 2

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/
```

Ruft das Backend auf. `-m 3` bricht nach drei Sekunden ab, `-w` gibt nur den Statuscode aus. Erwartet: `200`.

MAC Schritt 3

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.com/
```

Auch das Internet ist erreichbar. Erwartet: `200`. Noch gibt es keine Regeln.

6.3 Schritt 1: Alles sperren außer DNS

DATEI `security/netpol/01-default-deny.yaml`

```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: default-deny
  namespace: demo
spec:
  description: Alles sperren, nur DNS bleibt erlaubt
  endpointSelector: {}
  ingress:
    - {}
  egress:
    - toEndpoints:
        - matchLabels:
            io.kubernetes.pod.namespace: kube-system
            k8s-app: kube-dns
      toPorts:
        - ports:
            - port: "53"
              protocol: ANY
          rules:
            dns:
              - matchPattern: "*"
```

`endpointSelector: {}` wählt alle Pods im Namespace. `ingress: [{}]` ist eine leere Regel: Sie erlaubt nichts, schaltet aber Default-Deny für eingehenden Verkehr ein. Beim `egress` bleibt nur DNS erlaubt, sonst könnte kein Pod mehr Namen auflösen. Die `dns`-Regel lässt Cilium die Anfragen mitlesen, das brauchst du gleich für Hostnamen.

MAC Schritt 4

```
kubectl apply -f security/netpol/01-default-deny.yaml
```

Ab sofort ist `demo` abgeriegelt.

MAC Schritt 5

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/
```

Erwartet: `000` und ein Timeout. Der Name wird aufgelöst, die Verbindung aber still verworfen.

MAC Schritt 6

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  hubble observe --namespace demo --verdict DROPPED --last 10
```

Hubble zeigt dir genau, was blockiert wurde: Quelle, Ziel, Port und Grund.

DAS SOLLTEST DU SEHEN

Eine Zeile zeigt `demo/frontend-... -> demo/backend-...:8080`, dazu `Policy denied` und `DROPPED`. Das ist dein wichtigstes Werkzeug bei Netzwerkproblemen: Statt zu raten, schaust du einfach nach.

6.4 Schritt 2: Gezielt freigeben

Für eine Verbindung braucht es zwei Regeln: Das Frontend muss senden dürfen, und das Backend muss empfangen dürfen.

DATEI `security/netpol/02-frontend-backend.yaml`

```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: frontend-darf-senden
  namespace: demo
spec:
  endpointSelector:
    matchLabels:
      app: frontend
  egress:
    - toEndpoints:
        - matchLabels:
            app: backend
      toPorts:
        - ports:
            - port: "8080"
              protocol: TCP
---
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: backend-darf-empfangen
  namespace: demo
```

```
spec:
  endpointSelector:
    matchLabels:
      app: backend
  ingress:
    - fromEndpoints:
        - matchLabels:
            app: frontend
      toPorts:
        - ports:
            - port: "8080"
              protocol: TCP
```

Policies addieren sich. Die Sperre bleibt bestehen, diese Regeln fügen nur Ausnahmen hinzu. Ohne Namespace-Angabe gelten die Labels nur im eigenen Namespace.

MAC Schritt 7

```
kubectl apply -f security/netpol/02-frontend-backend.yaml
```

Öffnet genau diese eine Verbindung.

MAC Schritt 8

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/
```

Erwartet: `200`.

MAC Schritt 9

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.com/
```

Erwartet: `000`. Das Internet bleibt gesperrt.

6.5 Schritt 3: Regeln auf HTTP-Ebene

Cilium kann auch HTTP verstehen. Damit erlaubst du nicht nur einen Port, sondern konkrete Pfade. Ändere dafür die Regel `backend-darf-empfangen`:

DATEI `security/netpol/03-http-regel.yaml`

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: backend-darf-empfangen
  namespace: demo
spec:
  endpointSelector:
    matchLabels:
      app: backend
  ingress:
    - fromEndpoints:
        - matchLabels:
            app: frontend
```

```
    toPorts:
      - ports:
          - port: "8080"
            protocol: TCP
        rules:
          http:
            - method: GET
              path: "/"
```

MAC Schritt 10

```
kubectl apply -f security/netpol/03-http-regel.yaml
```

Der Name ist gleich, also ersetzt dieser Befehl die Regel.

MAC Schritt 11

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/
```

Erwartet: `200`.

MAC Schritt 12

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/admin
```

Erwartet: `403`. Der Unterschied zu vorher ist wichtig: Die Verbindung kommt zustande, aber Ciliums Proxy lehnt die Anfrage ab.

MAC Schritt 13

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' \
  -X DELETE http://backend:8080/
```

Erwartet: `403`. Nur `GET` ist erlaubt.

6.6 Schritt 4: Ausgang nach Hostname erlauben

Oft muss eine Anwendung einen bestimmten Dienst im Internet erreichen. IP-Adressen ändern sich, deshalb gibst du den Namen an.

DATEI `security/netpol/04-fqdn.yaml`

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: frontend-darf-example
  namespace: demo
spec:
  endpointSelector:
    matchLabels:
      app: frontend
  egress:
```

```
    - toFQDNs:
        - matchName: example.com
      toPorts:
        - ports:
            - port: "443"
              protocol: TCP
```

Das funktioniert nur dank der `dns`-Regel aus Schritt 1: Cilium liest die DNS-Antwort mit und merkt sich die IP-Adressen zu `example.com`.

MAC Schritt 14

```
kubectl apply -f security/netpol/04-fqdn.yaml
```

Lädt die Regel.

MAC Schritt 15

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.com/
```

Erwartet: `200`.

MAC Schritt 16

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.org/
```

Erwartet: `000`. Nur der genannte Host ist erlaubt.

6.7 Was weiterhin durchkommt

STOLPERFALLE

Der Kubelet prüft Pods weiter per Health-Check, obwohl alles gesperrt ist. Cilium erlaubt Verkehr vom eigenen Node standardmäßig. Pods, die den API-Server brauchen, musst du aber ausdrücklich freigeben: mit `egress` und `toEntities: [kube-apiserver]`. Ohne diese Regel scheitern Operatoren still.

6.8 Wie du das auf echte Namespaces überträgst

Sperre nie einen ganzen Cluster auf einen Schlag. Gehe Namespace für Namespace vor:

1. Lass den Dienst unter Last laufen.
2. Schaue in Hubble oder Grafana, welche Verbindungen es wirklich gibt.
3. Schreibe die Regeln passend dazu.
4. Schalte Default-Deny ein und beobachte `--verdict DROPPED`.

Die Datei `01-default-deny.yaml` kann als Vorlage dienen. `kube-system` bleibt dabei außen vor, denn dort läuft die Grundversorgung.

PROBIER ES SELBST

Sichere den Namespace `falco-lab` aus Kapitel 7, sobald du ihn hast: nur DNS und Ausgang zu `dl-cdn.alpinelinux.org`. Dann funktioniert `apk add` weiter, aber sonst nichts.

WAS DU GELERNT HAST

- Sobald eine Policy einen Pod auswählt, gilt für diese Richtung Default-Deny.
- Eine Verbindung braucht beide Seiten: Egress beim Sender, Ingress beim Empfänger.
- `hubble observe --verdict DROPPED` ersetzt das Raten.
- Cilium versteht auch HTTP und Hostnamen, nicht nur Ports.
- Ein Timeout (`000`) und `403` bedeuten verschiedene Dinge: Sperre auf Netzwerk- oder auf Anwendungsebene.

KAPITEL 7

Falco: Verdächtiges erkennen

Falco beobachtet Systemaufrufe auf dem Node und meldet, wenn sich ein Container verdächtig verhält.

Dauer: 60 Minuten **Voraussetzung:** Kapitel 4

7.1 Die Alarmanlage im Kernel

Bisher hast du verhindert. Was aber, wenn jemand durchkommt? Falco sitzt per eBPF im Kernel und sieht jeden Systemaufruf jedes Containers. Es vergleicht sie mit Regeln und meldet Treffer.

Falco sieht	Falco sieht nicht
Eine Shell startet im Container	Den Inhalt einer Verschlüsselung
Ein Prozess liest `/etc/shadow`	Ob eine Aktion fachlich erlaubt ist
Ein neues Programm wird ausgeführt	Was außerhalb des Nodes passiert

WARUM DAS WICHTIG IST

Falco blockiert nichts, es meldet nur. Deshalb ergänzt es Cilium und PSA: Was die Prävention durchlässt, sieht Falco. Eine Regel, die nie anschlägt, kann auch nichts stören. Eine laute Regel dagegen sorgt dafür, dass echte Alarme untergehen. Das Feintuning ist deshalb die eigentliche Arbeit.

7.2 Installieren

Falco schreibt seine Meldungen als JSON auf die Standardausgabe. Fürs Erste liest du sie mit `kubectl logs`. In Kapitel 10 sammelt Alloy sie automatisch ein und bringt sie nach Loki. Du brauchst also kein weiteres Zusatzprogramm und sparst RAM.

DATEI `falco/values.yaml`

```yaml
# Zeilen sofort ausgeben, nicht puffern
tty: true

driver:
  # Moderner eBPF-Treiber, kein Kernel-Modul nötig
  kind: modern_ebpf
```

```
falco:
  json_output: true
  json_include_output_property: true

resources:
  requests:
    cpu: 100m
    memory: 256Mi
  limits:
    memory: 768Mi

customRules:
  eigene-regeln.yaml: |-
    - rule: Paketmanager im Container
      desc: Jemand installiert Software in einem laufenden Container
      condition: >
        spawned_process and container
        and proc.name in (apt, apt-get, apk, dnf, yum)
      output: >
        Paketmanager gestartet (user=%user.name cmd=%proc.cmdline
        pod=%k8s.pod.name ns=%k8s.ns.name
        image=%container.image.repository)
      priority: WARNING
      tags: [container, mitre_persistence]
```

`modern_ebpf` läuft auf deinem Kernel 6.8 ohne Kompilieren. Den k3s-Socket `/run/k3s/containerd/containerd.sock` kennt das Chart von Haus aus, du musst ihn nicht eintragen. So bekommt Falco Container-Namen und Image-Infos.

MAC Schritt 1

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm repo update
```

Trägt das Repository ein.

MAC Schritt 2

```
helm upgrade --install falco falcosecurity/falco \
  -n falco --version 9.2.2 \
  -f falco/values.yaml --wait --timeout 5m
```

Der Namespace `falco` existiert seit Kapitel 4 mit der Stufe `privileged`. Hättest du ihn nicht angelegt, würde der clusterweite Standard Falco ablehnen.

MAC Schritt 3

```
kubectl logs -n falco ds/falco -c falco --tail=20
```

Prüft den Start. Du solltest sehen, dass Regeln geladen und die eBPF-Probe aktiv ist. Fehler in deinen eigenen Regeln stehen ebenfalls hier.

7.3 Einen Angriff simulieren

Du brauchst ein Opfer, das als root läuft. In `demo` geht das wegen `restricted` nicht. Dieser Unterschied ist schon die erste Lektion.

MAC Schritt 4

```
kubectl create namespace falco-lab
```

Ein Namespace mit dem Standard `baseline`, hier darf root laufen.

MAC Schritt 5

```
kubectl run opfer -n falco-lab --image=nginx:1.29-alpine
kubectl wait pod/opfer -n falco-lab --for=condition=Ready
```

Startet das Opfer und wartet, bis es bereit ist.

Öffne ein zweites Terminal und verfolge Falco live:

MAC Schritt 6

```
kubectl logs -n falco ds/falco -c falco -f --tail=0
```

`-f` folgt dem Log, `--tail=0` zeigt nur Neues. Lass dieses Terminal offen.

Zurück im ersten Terminal führst du typische Angreifer-Aktionen aus:

MAC Schritt 7

```
kubectl exec -it opfer -n falco-lab -- sh
```

Öffnet eine Shell im Container. Schon das löst die Regel `Terminal shell in container` aus.

POD Schritt 8

```
cat /etc/shadow
```

Liest die Passwort-Hashes. Regel: `Read sensitive file untrusted`.

POD Schritt 9

```
apk add --no-cache curl
```

Installiert Software. Das löst deine eigene Regel `Paketmanager im Container` aus.

POD Schritt 10

```
exit
```

Verlässt den Container.

DAS SOLLTEST DU SEHEN

Im zweiten Terminal erscheinen JSON-Zeilen mit `rule`, `priority` und `output_fields`. Du siehst Pod, Namespace, Image und die genaue Befehlszeile. Eine fehlende Zeile bei `Paketmanager` weist meist auf einen Fehler in `eigene-regeln.yaml` hin, schau dann in die Start-Logs.

7.4 Fehlalarme abstellen

Stell dir vor, dein Team öffnet täglich Shells in `falco-lab`. Dann wird die Regel `Terminal shell in container` zum Rauschen. Du erweiterst sie mit einer Ausnahme, statt sie abzuschalten.

Ergänze in `falco/values.yaml` unter `customRules` eine zweite Datei:

DATEI `falco/values.yaml (Ausschnitt)`

```
customRules:
  eigene-regeln.yaml: |-
    # ... wie oben ...
  ausnahmen.yaml: |-
    - rule: Terminal shell in container
      condition: and not k8s.ns.name = "falco-lab"
      override:
        condition: append
```

`override: condition: append` hängt deinen Text an die bestehende Bedingung an. So bleibt die Originalregel erhalten, und du bekommst Updates weiterhin. Die Dateien werden in alphabetischer Reihenfolge nach den Standardregeln geladen.

MAC Schritt 11

```
helm upgrade falco falcosecurity/falco \
  -n falco --version 9.2.2 -f falco/values.yaml --wait
```

Lädt die neuen Regeln.

MAC Schritt 12

```
kubectl exec opfer -n falco-lab -it -- sh -c 'echo test'
```

Falco sollte jetzt schweigen. In allen anderen Namespaces meldet es Shells weiter.

STOLPERFALLE

Schränke Ausnahmen so eng wie möglich ein. Eine Ausnahme für einen ganzen Namespace ist für ein Lab in Ordnung. In Produktion nimmst du besser eine Kombination aus Image und Benutzer. Sonst baust du dem Angreifer einen blinden Fleck.

PROBIER ES SELBST

Schreibe eine Regel, die anschlägt, wenn ein Container eine Verbindung auf Port 4444 öffnet, den typischen Port von Reverse Shells. Tipp: Schau dir das Makro `outbound` und das Feld `fd.sport` in der Falco-Doku an. Teste mit `nc` im Pod `opfer`.

WAS DU GELERNT HAST

- Falco sieht Systemaufrufe, blockiert aber nichts.
- JSON auf der Standardausgabe reicht, Kapitel 10 sammelt es zentral ein.
- Das Feintuning ist die eigentliche Arbeit, denn laute Regeln verdecken echte Alarme.
- Mit `override: append` ergänzt du Regeln, ohne sie zu überschreiben.

KAPITEL 8

Trivy Operator: Regelmäßig scannen

Der Cluster prüft sich selbst auf bekannte Schwachstellen und Fehlkonfigurationen, und du wertest die Berichte aus.

Dauer: 45 Minuten **Voraussetzung:** Kapitel 4

8.1 Bekannte Schwächen finden

Ein Image, das heute sauber ist, kann morgen verwundbar sein, weil eine neue Lücke bekannt wird. Einmaliges Scannen reicht also nicht. Der Trivy Operator beobachtet den Cluster und scannt alles, was läuft. Die Ergebnisse legt er als Kubernetes-Objekte ab.

Bericht	Kurzname	Inhalt
`VulnerabilityReport`	`vuln`	Bekannte CVEs in Images
`ConfigAuditReport`	`configaudit`	Unsichere Pod-Einstellungen
`ExposedSecretReport`	`exposedsecret`	Passwörter, die im Image liegen
`RbacAssessmentReport`	`rbacassessment`	Zu weite Rollen
`InfraAssessmentReport`	`infraassessment`	Einstellungen der Nodes
`ClusterComplianceReport`	`compliance`	Prüfung gegen Standards wie CIS

WARUM DAS WICHTIG IST

Weil die Berichte Kubernetes-Objekte sind, gelten für sie RBAC, `kubectl` und Prometheus. Du brauchst keinen eigenen Scan-Server, und jedes Team sieht die Berichte seines Namespace.

8.2 Installieren

DATEI `trivy/values.yaml`

```yaml
# Kapitel 10 schaltet das ein, sobald Prometheus läuft.
serviceMonitor:
  enabled: false

trivy:
  # Nur Lücken zeigen, für die es eine Korrektur gibt
  ignoreUnfixed: true
  severity: HIGH,CRITICAL
```

```
operator:
  # Nur ein Scan gleichzeitig, um den Node zu schonen
  scanJobsConcurrentLimit: 1
  # Berichte nach 24 Stunden neu erstellen
  scannerReportTTL: 24h
```

`ignoreUnfixed` blendet Meldungen aus, gegen die du gerade nichts tun kannst. `scanJobsConcurrentLimit: 1` ist auf 8 GB RAM wichtig, der Standardwert 10 könnte deinen Node überlasten. `scannerReportTTL` legt fest, dass täglich neu gescannt wird.

MAC Schritt 1

```
helm repo add aqua https://aquasecurity.github.io/helm-charts/
helm repo update
```

Trägt das Repository ein.

MAC Schritt 2

```
helm upgrade --install trivy-operator aqua/trivy-operator \
  -n trivy-system --version 0.38.1 \
  -f trivy/values.yaml --wait --timeout 5m
```

Installiert den Operator in den vorbereiteten Namespace. `trivy-system` hat seit Kapitel 4 die Stufe `privileged`, weil der Node-Collector Host-Dateien prüft.

MAC Schritt 3

```
kubectl get jobs -n trivy-system
```

Der Operator legt für jeden Workload einen kurzlebigen Scan-Job an. Bei einem frischen Start dauert es eine Weile, bis alle durch sind, und die Schwachstellendatenbank wird einmalig geladen.

8.3 Ein altes Image finden lassen

MAC Schritt 4

```
kubectl create deployment altlast -n falco-lab --image=nginx:1.21.6
```

Startet ein Image von 2022. Es enthält reichlich bekannte Lücken. Der Namespace `falco-lab` erlaubt root.

MAC Schritt 5

```
kubectl get vuln -n falco-lab -o wide
```

Nach ein paar Minuten erscheint ein Bericht. Die Spalten zeigen die Anzahl pro Schweregrad. Solange die Liste leer ist, läuft der Scan noch.

MAC Schritt 6

```
kubectl get vuln -n falco-lab -o json | jq -r '
  .items[].report.vulnerabilities[]
  | select(.severity == "CRITICAL")
  | "\(.vulnerabilityID)  \(.resource)  fix: \(.fixedVersion)"
' | sort -u | head -20
```

Zeigt die kritischen Lücken mit dem Paket und der Version, die sie behebt.

DAS SOLLTEST DU SEHEN

Du siehst eine Liste von CVE-Nummern, etwa für `openssl` oder `zlib`, jeweils mit `fix:`-Version. Die genaue Liste ändert sich mit der Datenbank.

PROBIER ES SELBST

Aktualisiere das Deployment: `kubectl set image deploy/altlast -n falco-lab altlast=nginx:1.29-alpine`. Wie viele kritische Lücken meldet der neue Bericht? Dieses Vorgehen ist in der Praxis der häufigste Weg, Lücken zu schließen: nicht patchen, sondern ein neues Image ausrollen.

8.4 Fehlkonfigurationen finden

MAC Schritt 7

```
kubectl get configaudit -A -o wide
```

Zeigt pro Workload die Anzahl der Befunde. Dein Pod in `demo` sollte deutlich besser abschneiden als `opfer` in `falco-lab`, weil er `restricted` erfüllt.

MAC Schritt 8

```
kubectl get configaudit -n falco-lab -o json | jq -r '
  .items[].report.checks[]
  | select(.success == false)
  | "\(.severity)  \(.checkID)  \(.title)"
' | sort -u
```

Listet alle fehlgeschlagenen Prüfungen. Viele davon entsprechen genau den Regeln, die du aus Kapitel 4 kennst.

MAC Schritt 9

```
kubectl get rbacassessment -A -o wide
```

Bewertet deine Rollen. Hier siehst du, wie Trivy die Rechte aus Kapitel 3 einschätzt.

MAC Schritt 10

```
kubectl get exposedsecret -A
```

Zeigt Geheimnisse, die jemand fest in ein Image eingebaut hat. Diese Liste sollte leer sein.

STOLPERFALLE

Die Infra-Prüfungen erwarten oft Standardpfade von `kubeadm`. Bei k3s läuft der API-Server im k3s-Prozess, deshalb erzeugt `infraassessment` einige Fehlalarme. Prüfe solche Meldungen, bevor du handelst, und vergleiche sie mit dem k3s-Hardening-Guide.

8.5 Die Regel, die alles verbindet

Kapitel 5 und dieses Kapitel lassen sich verbinden: Trivy meldet, dass ein Image verwundbar ist, und Kyverno kann per `ImageValidatingPolicy` verlangen, dass nur geprüfte Images starten. Das führt zu signierten Images und ist ein gutes nächstes Lernprojekt.

WAS DU GELERNT HAST

- Scannen ist kein Einmal-Ereignis, denn neue Lücken treffen auch alte Images.
- Trivy legt Berichte als Kubernetes-Objekte ab, also gelten RBAC und `kubectl`.
- Auf kleinen Nodes begrenzt `scanJobsConcurrentLimit` die Last.
- Lücken schließt man meist durch ein neues Image, nicht durch einen Patch im Container.

KAPITEL 9

Audit-Logs: Wer hat was getan?

Der API-Server protokolliert jede wichtige Anfrage, und du wertest das Protokoll aus.

Dauer: 30 Minuten **Voraussetzung:** Kapitel 4

9.1 Die Überwachungskamera des API-Servers

Jede Änderung im Cluster läuft über den API-Server. Damit ist er der perfekte Ort für ein Protokoll. Das Audit-Log beantwortet nach einem Vorfall die zentralen Fragen: Wer hat wann welche Aktion an welchem Objekt ausgeführt, und von wo aus?

Bei k3s läuft der API-Server im k3s-Prozess, nicht als Pod. Das Audit-Log ist deshalb eine Datei auf dem Host. Noch liest du sie direkt dort, in Kapitel 10 bringt Alloy sie nach Loki.

9.2 Die vier Protokoll-Stufen

Stufe	Was protokolliert wird
`None`	Nichts
`Metadata`	Wer, was, wann, Ergebnis. Keine Inhalte.
`Request`	Zusätzlich der Inhalt der Anfrage
`RequestResponse`	Zusätzlich der Inhalt der Antwort

WARUM DAS WICHTIG IST

Bei Secrets darfst du nie `Request` oder `RequestResponse` verwenden, sonst stehen die Passwörter im Klartext im Log, und dein Log wird selbst zum Risiko. Die erste passende Regel gewinnt, die Reihenfolge in der Datei entscheidet also.

9.3 Die Audit-Policy schreiben

DATEI `security/audit-policy.yaml`

```
apiVersion: audit.k8s.io/v1
kind: Policy
omitStages:
  - RequestReceived
rules:
```

```
  # Lärm weglassen: Gesundheitschecks und Metriken
  - level: None
    nonResourceURLs:
      - /healthz*
      - /readyz*
      - /livez*
      - /metrics
  # Events und Leader-Leases feuern ständig
  - level: None
    resources:
      - group: ""
        resources: [events]
      - group: coordination.k8s.io
        resources: [leases]
  # Inhalte von Secrets nie ins Log schreiben
  - level: Metadata
    resources:
      - group: ""
        resources: [secrets, configmaps]
      - group: authentication.k8s.io
        resources: [tokenreviews]
  # Rechteänderungen vollständig festhalten
  - level: RequestResponse
    verbs: [create, update, patch, delete]
    resources:
      - group: rbac.authorization.k8s.io
  # Alles andere nur als Metadaten
  - level: Metadata
```

Ohne die beiden `None`-Regeln am Anfang füllt sich das Log mit wertlosen Einträgen. `omitStages` lässt die erste Phase jeder Anfrage weg, sonst stünde jede Aktion doppelt im Log.

9.4 Auf dem Node aktivieren

MAC — Schritt 1

```
scp security/audit-policy.yaml \
  <user>@172.16.0.136:/tmp/audit.yaml
```

Bringt die Policy auf den Node.

Öffne jetzt `k3s/config.yaml` und kommentiere auch die Audit-Zeilen ein. Die `admission-control`-Zeile ist seit Kapitel 4 aktiv.

DATEI `k3s/config.yaml (Ausschnitt)`

```
kube-apiserver-arg:
  - "admission-control-config-file=/var/lib/rancher/k3s/server/psa.yaml"
  - "audit-policy-file=/var/lib/rancher/k3s/server/audit.yaml"
  - "audit-log-path=/var/lib/rancher/k3s/server/logs/audit.log"
  - "audit-log-maxage=30"
  - "audit-log-maxbackup=10"
  - "audit-log-maxsize=100"
```

`maxsize` rotiert die Datei bei 100 MB, `maxbackup` behält zehn alte Dateien, `maxage` löscht nach 30 Tagen. Das begrenzt den Platzbedarf auf rund 1,1 GB.

MAC Schritt 2

```
scp k3s/config.yaml <user>@172.16.0.136:/tmp/config.yaml
ssh <user>@172.16.0.136
```

Bringt die geänderte Konfiguration auf den Node und öffnet die Sitzung.

NODE Schritt 3

```
sudo install -m 600 -o root -g root /tmp/audit.yaml \
  /var/lib/rancher/k3s/server/audit.yaml
```

Die Policy muss vor dem Neustart an ihrem Platz liegen, sonst startet der API-Server nicht.

NODE Schritt 4

```
sudo install -m 600 -o root -g root /tmp/config.yaml \
  /etc/rancher/k3s/config.yaml
rm /tmp/audit.yaml /tmp/config.yaml
```

Setzt die neue Konfiguration ein und räumt `/tmp` auf.

NODE Schritt 5

```
sudo systemctl restart k3s
```

k3s liest `config.yaml` nur beim Start. Die Pods laufen während des Neustarts weiter.

NODE Schritt 6

```
sudo tail -n 1 /var/lib/rancher/k3s/server/logs/audit.log | jq .
```

Zeigt den letzten Eintrag lesbar an. Lies ihn einmal ganz durch: `user`, `verb`, `objectRef`, `sourceIPs`, `responseStatus`.

STOLPERFALLE

Startet k3s nicht mehr, ist meist die Policy-Datei schuld. `sudo journalctl -u k3s -n 50` nennt den Fehler. Zur Not kommentierst du die Zeilen in `/etc/rancher/k3s/config.yaml` aus und startest neu.

9.5 Das Log auswerten

Lass die SSH-Sitzung offen und öffne ein zweites Terminal auf dem Laptop.

MAC Schritt 7

```
kubectl create secret generic audit-test \
  --from-literal=wert=streng-geheim
```

Erzeugt ein Test-Secret, das du gleich im Log wiederfinden willst.

MAC Schritt 8

```
kubectl get secret audit-test -o yaml
```

Liest es, damit auch ein Lesezugriff im Log steht.

NODE Schritt 9

```
sudo jq -c 'select(.objectRef.name == "audit-test")
  | {verb, user: .user.username, code: .responseStatus.code}' \
  /var/lib/rancher/k3s/server/logs/audit.log
```

`jq` liest das Log als Strom von JSON-Objekten. `select` filtert, die zweite Stufe baut eine kurze Zeile.

DAS SOLLTEST DU SEHEN

Du siehst Einträge mit `create` und `get` und deinem Admin-Nutzer.

NODE Schritt 10

```
sudo grep -c streng-geheim \
  /var/lib/rancher/k3s/server/logs/audit.log
```

Muss `0` ausgeben: Der geheime Wert steht **nicht** im Log. Genau das hat die `Metadata`-Regel bewirkt.

MAC Schritt 11

```
kubectl delete secret audit-test
```

Räumt das Test-Objekt auf.

9.6 Nützliche Auswertungen

Wer außer Systemkomponenten hat Secrets gelesen?

NODE Schritt 12

```
sudo jq -c 'select(.objectRef.resource == "secrets"
    and (.verb == "get" or .verb == "list")
    and (.user.username | startswith("system:") | not))
  | {verb, user: .user.username, name: .objectRef.name}' \
  /var/lib/rancher/k3s/server/logs/audit.log
```

Die wichtigste Abfrage für den Alltag. In Kapitel 10 schreibst du sie in LogQL neu.

Welche Anfragen wurden abgelehnt?

NODE Schritt 13

```
sudo jq -c 'select(.responseStatus.code == 403)
  | {verb, user: .user.username, res: .objectRef.resource}' \
  /var/lib/rancher/k3s/server/logs/audit.log
```

Abgelehnte Anfragen zeigen oft Fehlkonfigurationen, manchmal aber auch einen Angreifer, der sich umsieht.

NODE Schritt 14

```
exit
```

Beendet die SSH-Sitzung.

PROBIER ES SELBST

Führe `kubectl get secrets -A --as=system:anonymous` aus. Was antwortet der Cluster, und was steht danach im Audit-Log? Starte dann einen normalen `nginx`-Pod in einem neuen Namespace und suche im Log die Annotation `pod-security.kubernetes.io/audit-violations`. Das ist der Audit-Modus von PSA aus Kapitel 4.

WAS DU GELERNT HAST

- Das Audit-Log beantwortet nach einem Vorfall: wer, was, wann, von wo.
- Die erste passende Regel gewinnt, daher stehen die `None`-Regeln oben.
- Secrets gehören nur auf Stufe `Metadata` ins Log.
- Bei k3s ist das Audit-Log eine Host-Datei. Solange es nur dort liegt, kann ein Angreifer es löschen.

KAPITEL 10

Zentrale Sicht: Grafana und Loki

Alle Schichten melden an einen Ort: Audit-Log, Falco, Trivy und Hubble landen in Grafana.

Dauer: 90 Minuten **Voraussetzung:** Kapitel 7, 8 und 9

10.1 Was du aufbaust

Bisher liegen alle Erkenntnisse verstreut: im Audit-Log auf dem Node, in den Falco-Logs, in Trivy-Objekten. Jetzt führst du sie zusammen.

Komponente	Aufgabe
SeaweedFS	S3-Speicher für Logs und Traces, aus deinem Repo
Prometheus	Speichert Metriken
Loki	Speichert Logs
Alloy	Sammelt Logs auf dem Node ein
Grafana	Zeigt alles an einer Stelle

WARUM DAS WICHTIG IST

Logs, die nur auf dem Node liegen, kann ein Angreifer löschen. Sobald sie in Loki landen, liegen sie außerhalb seiner Reichweite im Objektspeicher. Genau deshalb ist Sichtbarkeit selbst eine Sicherheitsmaßnahme.

UND THANOS?

Dein Repo enthält auch Thanos für die Langzeit-Speicherung von Metriken. Für dieses Buch brauchst du es nicht, und es spart rund 0,5 GB RAM. Möchtest du es später ergänzen, folge dem README des Repos.

10.2 SeaweedFS starten

Der Namespace `monitoring` existiert seit Kapitel 4 mit der Stufe `privileged`.

MAC Schritt 1

```
./seaweedfs/create-secrets.sh
```

Dein Skript aus dem Repo. Es erzeugt das Secret `s3-config` mit dem ersten Konto, an das du gleich weitere anhängst.

STOLPERFALLE

`seaweedfs/seaweedfs.yaml` begrenzt den Speicher mit `-volume.max=16` auf 16 × 256 MB, also rund 4 GB. Für Loki und Tempo wird das eng. Erhöhe den Wert in der Datei auf `-volume.max=32`, bevor du sie anwendest. Deine Disk verkraftet die 8 GB.

MAC — Schritt 2

```
kubectl apply -f seaweedfs/seaweedfs.yaml
```

Legt PVC, Deployment und Service des S3-Speichers an.

MAC — Schritt 3

```
kubectl rollout status deploy/s3 -n monitoring
```

Wartet, bis der Pod bereit ist. `apply` kehrt sofort zurück, dieser Befehl nicht.

MAC — Schritt 4

```
kubectl exec -n monitoring deploy/s3 -- sh -c \
  "echo 's3.bucket.create -name loki' | weed shell -master=localhost:9333"
```

Legt den Bucket `loki` über die SeaweedFS-Verwaltungsshell an.

10.3 Ein eigenes S3-Konto für Loki

Jeder Dienst bekommt sein eigenes Konto mit Zugriff nur auf seinen Bucket. Wird ein Dienst kompromittiert, ist nur dieser Bucket betroffen.

MAC — Schritt 5

```
LOKI_ACCESS=$(openssl rand -hex 10)
LOKI_SECRET=$(openssl rand -hex 20)
```

Erzeugt zwei Zufallswerte und legt sie in Shell-Variablen. Sie erscheinen weder auf dem Bildschirm noch in einer Datei.

MAC — Schritt 6

```
ALT=$(kubectl get secret s3-config -n monitoring \
  -o jsonpath='{.data.s3\.json}' | base64 -d)
```

Liest die bestehende Kontenliste. Kubernetes speichert Secrets Base64-kodiert, `base64 -d` macht sie wieder lesbar. Der Backslash in `s3\.json` ist nötig, weil der Punkt sonst als Pfadtrenner gilt.

MAC Schritt 7

```
NEU=$(printf '%s' "$ALT" | jq \
  --arg a "$LOKI_ACCESS" --arg s "$LOKI_SECRET" \
  '.identities += [{
     name: "loki",
     credentials: [{accessKey: $a, secretKey: $s}],
     actions: ["Read:loki","Write:loki","List:loki","Tagging:loki"]
   }]')
```

`jq` hängt ein neues Konto an die Liste an. `--arg` übergibt die Werte sicher, statt sie in den Text einzusetzen. `actions` beschränkt das Konto auf den Bucket `loki`.

MAC Schritt 8

```
printf '%s' "$NEU" | kubectl create secret generic s3-config \
  -n monitoring --from-file=s3.json=/dev/stdin \
  --dry-run=client -o yaml | kubectl apply -f -
```

Baut lokal ein neues Secret (`--dry-run=client -o yaml`) und wendet es an. Die Daten laufen über die Standardeingabe, damit sie nicht in der Prozessliste auftauchen.

MAC Schritt 9

```
kubectl rollout restart deploy/s3 -n monitoring
kubectl rollout status deploy/s3 -n monitoring
```

SeaweedFS liest die Konten nur beim Start, also musst du neu starten.

MAC Schritt 10

```
kubectl create secret generic loki-s3 -n monitoring \
  --from-literal=S3_ACCESS_KEY="$LOKI_ACCESS" \
  --from-literal=S3_SECRET_KEY="$LOKI_SECRET"
```

Legt die Zugangsdaten für Loki ab. Die Schlüsselnamen werden später zu Umgebungsvariablen.

MAC Schritt 11

```
unset LOKI_ACCESS LOKI_SECRET ALT NEU
```

Entfernt die Geheimnisse aus deiner Shell.

10.4 Prometheus und Grafana installieren

Lege zuerst eine Werte-Datei an, die den Stack an k3s anpasst:

DATEI `monitoring/kps-k3s-values.yaml`

```
grafana:
  admin:
    existingSecret: grafana-admin
    userKey: admin-user
    passwordKey: admin-password
  additionalDataSources:
```

```
    - name: Loki
      uid: loki
      type: loki
      access: proxy
      url: http://loki.monitoring.svc.cluster.local:3100
      editable: false
    - name: Tempo
      uid: tempo
      type: tempo
      access: proxy
      url: http://tempo.monitoring.svc.cluster.local:3200
      editable: false
      jsonData:
        tracesToLogsV2:
          datasourceUid: loki

# Cilium ersetzt kube-proxy, also gibt es nichts zu überwachen.
kubeProxy:
  enabled: false
# Bei k3s laufen diese im k3s-Prozess, nicht als eigene Pods.
kubeControllerManager:
  enabled: false
kubeScheduler:
  enabled: false
kubeEtcd:
  enabled: false

prometheus:
  prometheusSpec:
    retention: 2d
    # Auch ServiceMonitors ohne Label "release: kps" einsammeln.
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    ruleSelectorNilUsesHelmValues: false
```

Die letzten drei Werte sind wichtig: Standardmäßig beachtet Prometheus nur Objekte seines eigenen Helm-Releases. Mit `false` findet es auch die ServiceMonitors von Trivy und Cilium und deine Regeln. Die Datenquelle Tempo liegt schon bereit, Tempo selbst kommt in Kapitel 11.

MAC Schritt 12

```
kubectl create secret generic grafana-admin -n monitoring \
  --from-literal=admin-user=admin \
  --from-literal=admin-password="$(openssl rand -hex 16)"
```

Legt ein zufälliges Grafana-Passwort an. Es steht in keiner Datei, nur im Cluster.

MAC Schritt 13

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update
```

Falls das Repository schon eingetragen ist, meldet Helm das nur. `update` holt die aktuellen Versionslisten.

MAC Schritt 14

```
helm upgrade --install kps \
  prometheus-community/kube-prometheus-stack \
  -n monitoring --version 92.18.0 \
  -f monitoring/kps-k3s-values.yaml \
  --wait --timeout 10m
```

Installiert Prometheus, Alertmanager, Grafana und die Exporter. Die Version entspricht deinem README.

10.5 Loki installieren

Das Loki-Chart ist in die Grafana-Community umgezogen. Das alte `grafana/loki` hängt auf Version 7.5.0 fest und wird nicht mehr aktuell gehalten.

DATEI `monitoring/loki-values.yaml`

```yaml
deploymentMode: Monolithic

loki:
  auth_enabled: false
  commonConfig:
    replication_factor: 1
  schemaConfig:
    configs:
      - from: "2026-01-01"
        store: tsdb
        object_store: s3
        schema: v13
        index:
          prefix: loki_index_
          period: 24h
  storage:
    type: s3
    bucketNames:
      chunks: loki
      ruler: loki
      admin: loki
    s3:
      endpoint: http://s3.monitoring.svc.cluster.local:8333
      region: us-east-1
      # Loki ersetzt das beim Start aus den Umgebungsvariablen.
      accessKeyId: ${S3_ACCESS_KEY}
      secretAccessKey: ${S3_SECRET_KEY}
      s3ForcePathStyle: true
      insecure: true
  compactor:
    retention_enabled: true
    delete_request_store: s3
  limits_config:
    retention_period: 168h

singleBinary:
  replicas: 1
  extraEnvFrom:
    - secretRef:
        name: loki-s3
  persistence:
    size: 5Gi
```

```
    storageClass: local-path
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      memory: 1Gi

# Auf einem kleinen Node sparen diese Abschaltungen viel RAM.
gateway:
  enabled: false
chunksCache:
  enabled: false
resultsCache:
  enabled: false
lokiCanary:
  enabled: false
test:
  enabled: false
backend:
  replicas: 0
read:
  replicas: 0
write:
  replicas: 0
```

`replication_factor: 1` ist Pflicht bei nur einer Instanz, sonst schlagen Anfragen fehl. `s3ForcePathStyle` ist für SeaweedFS nötig, und `retention_period: 168h` hält Logs sieben Tage. Mit `${...}` stehen keine Geheimnisse in der Datei. Das Chart setzt `-config.expand-env=true` selbst.

MAC

Schritt 15

```
helm repo add grafana-community \
  https://grafana-community.github.io/helm-charts
helm repo update
```

Das neue Zuhause der Charts für Loki und Tempo.

MAC

Schritt 16

```
helm upgrade --install loki grafana-community/loki \
  -n monitoring --version 21.0.0 \
  -f monitoring/loki-values.yaml --wait --timeout 5m
```

Installiert Loki als einen einzigen Prozess. Das reicht laut Doku für einige zehn GB pro Tag.

MAC

Schritt 17

```
kubectl logs -n monitoring statefulset/loki --tail=20
```

Prüft den Start. Bei S3-Problemen stehen hier Meldungen wie `AccessDenied`, dann stimmt das Konto nicht.

10.6 Alloy sammelt die Logs ein

Alloy läuft als DaemonSet, also einmal pro Node. Es liest die Container-Logs unter `/var/log/pods` und dein Audit-Log und schickt beides an Loki. Die Konfiguration trennst du von den Helm-Werten, damit du sie leicht ändern kannst.

DATEI `monitoring/config.alloy`

```
// Pods auf diesem Node finden
discovery.kubernetes "pods" {
  role = "pod"
  selectors {
    role  = "pod"
    field = "spec.nodeName=" + sys.env("HOSTNAME")
  }
}

// Labels setzen und den Pfad zur Logdatei bauen
discovery.relabel "pods" {
  targets = discovery.kubernetes.pods.targets

  rule {
    source_labels = ["__meta_kubernetes_namespace"]
    target_label  = "namespace"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_name"]
    target_label  = "pod"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_container_name"]
    target_label  = "container"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_uid",
                     "__meta_kubernetes_pod_container_name"]
    separator     = "/"
    replacement   = "/var/log/pods/*$1/*.log"
    target_label  = "__path__"
  }
}

local.file_match "pods" {
  path_targets = discovery.relabel.pods.output
}

loki.source.file "pods" {
  targets    = local.file_match.pods.targets
  forward_to = [loki.process.pods.receiver]
}

// Das CRI-Format von containerd auspacken
loki.process "pods" {
  stage.cri {}
  forward_to = [loki.write.default.receiver]
}

// Audit-Log des API-Servers aus Kapitel 9
local.file_match "audit" {
  path_targets = [{
    "__path__" = "/k3s-logs/audit.log",
    "job"      = "audit",
```

```
  }]
}

loki.source.file "audit" {
  targets    = local.file_match.audit.targets
  forward_to = [loki.write.default.receiver]
}

loki.write "default" {
  endpoint {
    url = "http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push"
  }
}
```

Lies die Datei von unten nach oben: `loki.write` ist das Ziel, jeder Block leitet mit `forward_to` weiter.

DATEI `monitoring/alloy-values.yaml`

```yaml
alloy:
  # Das Audit-Log gehört root und ist nur für root lesbar.
  securityContext:
    runAsUser: 0
    runAsGroup: 0
  configMap:
    create: false
    name: alloy-config
    key: config.alloy
  mounts:
    varlog: true
    extra:
      - name: k3s-logs
        mountPath: /k3s-logs
        readOnly: true

controller:
  type: daemonset
  volumes:
    extra:
      - name: k3s-logs
        hostPath:
          path: /var/lib/rancher/k3s/server/logs
          type: DirectoryOrCreate
```

Alloy muss hier als root laufen, das ist ein bewusster Kompromiss: Der Pod hat Lesezugriff auf den Host, dafür mountet er das Audit-Verzeichnis nur lesend. Das Chart gibt dem Pod auch die nötigen Rechte für `discovery.kubernetes`.

MAC Schritt 18

```
kubectl create configmap alloy-config -n monitoring \
  --from-file=config.alloy=monitoring/config.alloy
```

Legt die Konfiguration als ConfigMap ab. Bei Änderungen ergänzt du `--dry-run=client -o yaml | kubectl apply -f -`.

MAC Schritt 19

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

Alloy bleibt im Hauptrepository von Grafana.

MAC Schritt 20

```
helm upgrade --install alloy grafana/alloy \
  -n monitoring --version 1.15.0 \
  -f monitoring/alloy-values.yaml --wait
```

Startet Alloy als DaemonSet.

MAC Schritt 21

```
kubectl logs -n monitoring ds/alloy -c alloy --tail=20
```

Syntaxfehler in `config.alloy` stehen hier mit Zeilennummer.

10.7 Hubble und Trivy anbinden

Beide Dienste laufen schon. Jetzt, da Prometheus die ServiceMonitor-CRDs mitgebracht hat, können sie Metriken liefern.

DATEI `cilium/metrics-values.yaml`

```
prometheus:
  enabled: true
  serviceMonitor:
    enabled: true
operator:
  prometheus:
    enabled: true
    serviceMonitor:
      enabled: true
hubble:
  metrics:
    enabled:
      - dns
      - drop
      - tcp
      - flow
      - policy:sourceContext=workload-name;destinationContext=workload-name
    serviceMonitor:
      enabled: true
```

MAC Schritt 22

```
helm upgrade cilium oci://quay.io/cilium/charts/cilium \
  --version 1.20.4 -n kube-system \
  -f cilium/values.yaml -f cilium/metrics-values.yaml --wait
```

Führt beide Dateien zusammen. Die Agenten starten dabei neu, bestehende Verbindungen laufen meist weiter.

Ändere in `trivy/values.yaml` die Zeile unter `serviceMonitor` auf `enabled: true`.

MAC Schritt 23

```
helm upgrade trivy-operator aquasec/trivy-operator \
  -n trivy-system --version 0.38.1 \
  -f trivy/values.yaml --wait
```

Legt den ServiceMonitor für Trivy an.

Falco schreibt schon auf die Standardausgabe, dafür ist nichts weiter zu tun. Default-Deny aus Kapitel 6 sperrt Prometheus aber aus `demo` aus. Soll deine Anwendung Metriken liefern, erlaubst du das ausdrücklich:

DATEI `security/netpol/05-prometheus.yaml`

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: prometheus-darf-scrapen
  namespace: demo
spec:
  endpointSelector: {}
  ingress:
    - fromEndpoints:
        - matchLabels:
            io.kubernetes.pod.namespace: monitoring
            app.kubernetes.io/name: prometheus
```

Mit `io.kubernetes.pod.namespace` wählst du Pods aus einem anderen Namespace aus. Hier ist nur Prometheus gemeint, nicht der ganze Namespace.

MAC Schritt 24

```
kubectl apply -f security/netpol/05-prometheus.yaml
```

Lädt die Regel.

10.8 Grafana öffnen

MAC Schritt 25

```
kubectl get secret grafana-admin -n monitoring \
  -o jsonpath='{.data.admin-password}' | base64 -d; echo
```

Zeigt das Grafana-Passwort einmalig im Terminal an.

MAC Schritt 26

```
kubectl port-forward -n monitoring svc/kps-grafana 3000:80
```

Grafana ist nun unter http://localhost:3000 erreichbar. Anmeldung mit `admin`.

Öffne **Explore**, wähle die Datenquelle **Loki** und gib diese Abfrage ein:

```
{namespace="kube-system", container="coredns"}
```

DAS SOLLTEST DU SEHEN

Die Log-Zeilen von CoreDNS erscheinen. Wechselst du zur Datenquelle **Prometheus** und fragst `hubble_flows_processed_total` ab, siehst du die Hubble-Metriken.

10.9 Audit-Logs abfragen

Die `jq`-Abfragen aus Kapitel 9 schreibst du jetzt in LogQL. `| json` zerlegt jede Zeile, verschachtelte Felder wie `objectRef.name` werden dabei zu `objectRef_name`.

Wer außer Systemkomponenten hat Secrets gelesen?

```
{job="audit"} | json
  | objectRef_resource="secrets"
  | verb=~"get|list"
  | user_username!~"system:.*"
```

Welche Anfragen wurden abgelehnt?

```
{job="audit"} | json | responseStatus_code="403"
```

Wer hat RBAC-Regeln geändert?

```
{job="audit"} | json
  | objectRef_apiGroup="rbac.authorization.k8s.io"
  | verb=~"create|update|patch|delete"
```

10.10 Falco-Meldungen abfragen

`line_format` zeigt nur den Regelnamen:

```
{namespace="falco", container="falco"} | json | line_format "{{.rule}}"
```

Nur die ernsten Meldungen:

```
{namespace="falco", container="falco"}
  | json
  | priority =~ "Warning|Error|Critical|Alert|Emergency"
```

Nach Regel gezählt:

```
sum by (rule) (
  count_over_time(
    {namespace="falco", container="falco"} | json | rule != "" [1h]
  )
)
```

10.11 Trivy-Ergebnisse abfragen

Wähle die Datenquelle **Prometheus**. Prüfe zuerst unter **Explore**, ob `up{job=~".*trivy.*"}` den Wert `1` liefert.

```
sum by (severity) (trivy_image_vulnerabilities)
topk(5, sum by (namespace, image_repository)
  (trivy_image_vulnerabilities{severity="Critical"}))
sum by (severity) (trivy_resource_configaudits)
```

PROBIER ES SELBST

Schreibe eine LogQL-Abfrage, die alle Log-Zeilen mit dem Wort `error` aus dem Namespace `monitoring` zeigt. Tipp: Filter hängst du mit `|=` an den Label-Selektor an.

WAS DU GELERNT HAST

- Logs außerhalb des Nodes kann ein Angreifer nicht einfach löschen.
- Jeder Dienst bekommt sein eigenes S3-Konto, damit ein Einbruch begrenzt bleibt.
- Prometheus sammelt fremde ServiceMonitors erst ein, wenn du es erlaubst.
- Alle Schichten landen jetzt an einem Ort und lassen sich gemeinsam abfragen.

KAPITEL 11

Tempo: Dem Weg einer Anfrage folgen

Du speicherst Traces in Tempo und siehst, was während einer API-Anfrage passiert, bis hin zur Prüfung durch Kyverno.

Dauer: 60 Minuten **Voraussetzung:** Kapitel 10

11.1 Drei Arten von Signalen

Signal	Beantwortet	Speicher
Metriken	Wie viel, wie oft?	Prometheus
Logs	Was genau ist passiert?	Loki
Traces	Welchen Weg nahm eine Anfrage, und wo verging die Zeit?	Tempo

Ein Trace besteht aus **Spans**. Jeder Span ist ein Arbeitsschritt mit Start und Dauer.

WAS TEMPO FÜR DIE SICHERHEIT LEISTET

Tempo ist kein Sicherheitstool im engeren Sinn, und in deinem Cluster liefern Anwendungen erst Traces, wenn du sie instrumentierst. Nützlich ist es trotzdem: Der API-Server kann seine Anfragen als Traces liefern. Du siehst dann, wie lange etwa der Kyverno-Webhook eine Anfrage aufhält. Das hilft bei Störungen durch Admission-Webhooks und zeigt später in eigenen Anwendungen, welchen Weg eine verdächtige Anfrage genommen hat.

11.2 Bucket und S3-Konto

Das Muster kennst du aus Kapitel 10.

MAC Schritt 1

```
kubectl exec -n monitoring deploy/s3 -- sh -c \
  "echo 's3.bucket.create -name tempo' | weed shell -master=localhost:9333"
```

Legt den Bucket `tempo` an.

MAC Schritt 2

```
TEMPO_ACCESS=$(openssl rand -hex 10)
TEMPO_SECRET=$(openssl rand -hex 20)
ALT=$(kubectl get secret s3-config -n monitoring \
  -o jsonpath='{.data.s3\.json}' | base64 -d)
```

Erzeugt die Zugangsdaten und liest die Kontenliste.

MAC Schritt 3

```
NEU=$(printf '%s' "$ALT" | jq \
  --arg a "$TEMPO_ACCESS" --arg s "$TEMPO_SECRET" \
  '.identities += [{
     name: "tempo",
     credentials: [{accessKey: $a, secretKey: $s}],
     actions: ["Read:tempo","Write:tempo","List:tempo","Tagging:tempo"]
   }]')
```

Hängt das Konto an. Tempo darf nur in seinen eigenen Bucket.

MAC Schritt 4

```
printf '%s' "$NEU" | kubectl create secret generic s3-config \
  -n monitoring --from-file=s3.json=/dev/stdin \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl rollout restart deploy/s3 -n monitoring
kubectl rollout status deploy/s3 -n monitoring
```

Schreibt die Liste zurück und startet SeaweedFS neu.

MAC Schritt 5

```
kubectl create secret generic tempo-s3 -n monitoring \
  --from-literal=S3_ACCESS_KEY="$TEMPO_ACCESS" \
  --from-literal=S3_SECRET_KEY="$TEMPO_SECRET"
unset TEMPO_ACCESS TEMPO_SECRET ALT NEU
```

Legt die Zugangsdaten ab und räumt die Shell auf.

11.3 Tempo installieren

DATEI `monitoring/tempo-values.yaml`

```
tempo:
  retention: 168h
  storage:
    trace:
      backend: s3
      s3:
        bucket: tempo
        endpoint: s3.monitoring.svc.cluster.local:8333
        # Tempo ersetzt das beim Start aus den Umgebungsvariablen.
        access_key: ${S3_ACCESS_KEY}
        secret_key: ${S3_SECRET_KEY}
        insecure: true
        forcepathstyle: true
      wal:
```

```
        path: /var/tempo/wal
  extraArgs:
    config.expand-env: "true"
  extraEnvFrom:
    - secretRef:
        name: tempo-s3
  resources:
    requests:
      cpu: 50m
      memory: 256Mi
    limits:
      memory: 768Mi

persistence:
  enabled: true
  size: 2Gi
  storageClassName: local-path
```

Anders als bei Loki musst du `config.expand-env` hier selbst setzen. Die Endpoint-Angabe steht ohne `http://`, das `insecure: true` regelt das Protokoll. Der `wal` liegt lokal und puffert Traces, bevor sie ins S3 wandern.

MAC Schritt 6

```
helm upgrade --install tempo grafana-community/tempo \
  -n monitoring --version 3.2.1 \
  -f monitoring/tempo-values.yaml --wait --timeout 5m
```

Installiert Tempo als einzelnen Prozess, passend zu deinem einen Node.

MAC Schritt 7

```
kubectl logs -n monitoring statefulset/tempo --tail=20
```

Prüft den Start. `AccessDenied` weist wieder auf das Konto hin.

11.4 Alloy nimmt Traces an

Alloy sammelt nicht nur Logs ein, es nimmt auch Traces per OTLP an und reicht sie an Tempo weiter. So gibt es für alle Signale nur einen Sammelpunkt. Hänge diese Blöcke an `monitoring/config.alloy` an:

DATEI `monitoring/config.alloy (am Ende ergänzen)`

```
// Traces per OTLP annehmen
otelcol.receiver.otlp "default" {
  grpc {
    endpoint = "0.0.0.0:4317"
  }
  output {
    traces = [otelcol.processor.batch.default.input]
  }
}

// In Paketen senden, das schont das Netz
```

```
otelcol.processor.batch "default" {
  output {
    traces = [otelcol.exporter.otlp.tempo.input]
  }
}

otelcol.exporter.otlp "tempo" {
  client {
    endpoint = "tempo.monitoring.svc.cluster.local:4317"
    tls {
      insecure = true
    }
  }
}
```

Der API-Server läuft auf dem Host und kennt keine Cluster-DNS-Namen. Er braucht daher eine feste IP. Ergänze dafür `monitoring/alloy-values.yaml`:

DATEI `monitoring/alloy-values.yaml (ergänzen)`

```yaml
alloy:
  # ... wie bisher, zusätzlich:
  extraPorts:
    - name: otlp-grpc
      port: 4317
      targetPort: 4317
      protocol: TCP

# Neuer Block auf oberster Ebene
service:
  clusterIP: 10.43.0.50
```

`10.43.0.50` liegt im Service-Netz und ist nur im Cluster und auf dem Node erreichbar, nicht aus deinem Heimnetz. Einen `hostPort` auf `127.0.0.1` kann Cilium in dieser Betriebsart nicht bedienen, deshalb dieser Weg.

MAC — Schritt 8

```
kubectl create configmap alloy-config -n monitoring \
  --from-file=config.alloy=monitoring/config.alloy \
  --dry-run=client -o yaml | kubectl apply -f -
```

Aktualisiert die ConfigMap.

MAC — Schritt 9

```
kubectl delete svc alloy -n monitoring
helm upgrade alloy grafana/alloy \
  -n monitoring --version 1.15.0 \
  -f monitoring/alloy-values.yaml --wait
```

Die ClusterIP eines Service lässt sich nicht ändern. Deshalb löschst du ihn, und Helm legt ihn mit der neuen IP an.

MAC Schritt 10

```
kubectl get svc alloy -n monitoring
kubectl logs -n monitoring ds/alloy -c alloy --tail=20
```

Die Spalte `CLUSTER-IP` zeigt `10.43.0.50`, und im Log stehen keine Fehler.

11.5 Der erste Trace

Das Testprogramm `telemetrygen` erzeugt künstliche Traces.

MAC Schritt 11

```
kubectl run telemetrygen -n monitoring --rm -it --restart=Never \
  --image=ghcr.io/open-telemetry/opentelemetry-collector-contrib/telemetrygen:v0.164.1 \
  -- traces --otlp-endpoint=alloy.monitoring.svc.cluster.local:4317 \
  --otlp-insecure --traces=5
```

Schickt fünf Traces an Alloy. Sie gehen weiter an Tempo.

Öffne in Grafana **Explore**, wähle die Datenquelle **Tempo** und den Abfragetyp **TraceQL**:

```
{ resource.service.name = "telemetrygen" }
```

DAS SOLLTEST DU SEHEN

Du siehst eine Liste von Traces. Ein Klick zeigt die Spans als Wasserfall. Falls nichts erscheint, warte eine Minute und prüfe die Alloy-Logs auf Verbindungsfehler zu Tempo.

11.6 Der API-Server liefert Traces

DATEI `security/tracing.yaml`

```
apiVersion: apiserver.config.k8s.io/v1
kind: TracingConfiguration
endpoint: 10.43.0.50:4317
# 100000 von einer Million: jede zehnte Anfrage
samplingRatePerMillion: 100000
```

Für das Lab ist jede zehnte Anfrage gut sichtbar. Im Dauerbetrieb senkst du den Wert, denn Traces kosten Speicher. Die Verbindung läuft laut Kubernetes-Doku unverschlüsselt und kann kein TLS, sie darf also nur intern laufen.

MAC Schritt 12

```
scp security/tracing.yaml <user>@172.16.0.136:/tmp/tracing.yaml
```

Bringt die Datei auf den Node.

Ergänze in `k3s/config.yaml` unter `kube-apiserver-arg` eine Zeile:

DATEI `k3s/config.yaml (ergänzen)`

```
  - "tracing-config-file=/var/lib/rancher/k3s/server/tracing.yaml"
```

MAC Schritt 13

```
scp k3s/config.yaml <user>@172.16.0.136:/tmp/config.yaml
ssh <user>@172.16.0.136
```

Überträgt die Konfiguration und öffnet die Sitzung.

NODE Schritt 14

```
sudo install -m 600 -o root -g root /tmp/tracing.yaml \
  /var/lib/rancher/k3s/server/tracing.yaml
sudo install -m 600 -o root -g root /tmp/config.yaml \
  /etc/rancher/k3s/config.yaml
rm /tmp/tracing.yaml /tmp/config.yaml
sudo systemctl restart k3s
```

Setzt beide Dateien ein und startet k3s neu.

NODE Schritt 15

```
exit
```

Zurück zum Laptop.

MAC Schritt 16

```
kubectl get pods -A > /dev/null
kubectl run trace-test -n falco-lab --image=nginx:1.29-alpine
kubectl delete pod trace-test -n falco-lab
```

Erzeugt API-Anfragen. Wiederhole das ein paarmal, denn nur jede zehnte wird aufgezeichnet.

```
{ resource.service.name = "apiserver" }
```

Öffne einen Trace zum Anlegen eines Pods und betrachte die Spans. Webhook-Aufrufe suchst du gezielt so:

```
{ resource.service.name = "apiserver" && name =~ ".*webhook.*" }
```

DAS SOLLTEST DU SEHEN

Du findest Spans für Webhook-Aufrufe, darunter den von Kyverno, mit ihrer Dauer. Die genauen Span-Namen hängen von der Kubernetes-Version ab. Findest du nichts, entferne den Namensfilter und suche im Wasserfall.

STOLPERFALLE

Traces können interne URLs und Objektnamen enthalten. Behandle Tempo daher wie Logs: kein Zugriff von außen, und der OTLP-Port nimmt ohne Anmeldung alles an. Jeder Pod im Cluster könnte also Traces einliefern. In Namespaces mit Default-Deny ist das gesperrt, bis du es erlaubst.

11.7 Von Traces zu Logs springen

Die Datenquelle hat `tracesToLogsV2` aus Kapitel 10 gesetzt. Klicke in einem Span auf **Logs for this span**, dann öffnet Grafana passende Loki-Zeilen. Auf diese Weise kommst du von „diese Anfrage war langsam“ zu „das stand dabei im Log“.

PROBIER ES SELBST

Stoppe Kyverno mit `kubectl scale deploy -n kyverno kyverno-admission-controller --replicas=0` und lege einen Pod in `falco-lab` an. Was passiert, und wie sieht der Trace aus? Skaliere danach unbedingt wieder auf `1` hoch.

WAS DU GELERNT HAST

- Metriken zählen, Logs erzählen, Traces zeigen den Weg.
- Alloy ist der gemeinsame Sammelpunkt für Logs und Traces.
- Auch der API-Server liefert Traces, darin siehst du Admission-Webhooks.
- Dienste auf dem Host brauchen eine feste IP, weil sie kein Cluster-DNS kennen.

KAPITEL 12

Alles zusammenführen

Aus Einzeltools wird eine Architektur: ein Dashboard, Alarme und eine Angriffsübung über alle Schichten.

Dauer: 90 Minuten **Voraussetzung:** Kapitel 1 bis 11

12.1 Das Sicherheits-Dashboard

Ein gutes Dashboard beantwortet in zehn Sekunden: Ist gerade etwas los? Lege in Grafana unter **Dashboards > New** ein neues Dashboard an und baue diese Panels. Du lernst dabei mehr, als wenn du ein fertiges JSON importierst.

Panel	Quelle	Abfrage
Falco nach Regel	Loki	Abfrage siehe unten
Blockierte Pakete	Prometheus	`sum by (reason) (rate(hubble_drop_total[5m]))`
Kritische CVEs	Prometheus	`sum(trivy_image_vulnerabilities{severity="Critical"})`
Fehlkonfigurationen	Prometheus	`sum by (severity) (trivy_resource_configaudits)`
Abgelehnte API-Anfragen	Loki	Abfrage siehe unten
Langsame Webhooks	Tempo	Abfrage siehe unten

Falco nach Regel (Panel-Typ **Bar chart**):

```
sum by (rule) (
  count_over_time(
    {namespace="falco", container="falco"}
    | json | priority =~ "Warning|Error|Critical" [1h]
  )
)
```

Abgelehnte API-Anfragen (Panel-Typ **Time series**):

```
sum(
  count_over_time(
    {job="audit"} | json | responseStatus_code="403" [5m]
  )
)
```

Webhook-Aufrufe über 100 Millisekunden (Panel-Typ **Table**):

```
{ resource.service.name = "apiserver" && name =~ ".*webhook.*"
  && duration > 100ms }
```

Menschliche Zugriffe auf Secrets (Panel-Typ **Logs**):

```
{job="audit"} | json
  | objectRef_resource="secrets"
  | user_username !~ "system:.*"
```

Dazu passt der Stand aus Kyverno, den du per `kubectl get polr -A` siehst. Möchtest du ihn im Dashboard, aktiviere in `kyverno/values.yaml` bei `admissionController` und `reportsController` den Wert `serviceMonitor.enabled: true`.

WARUM DAS WICHTIG IST

Exportiere das fertige Dashboard als JSON nach `security/dashboard.json`. So überlebt es im Repo einen Neuaufbau der VM.

12.2 Alarme

Ein Dashboard hilft nur, wenn jemand hinschaut. Für die wichtigsten Fälle legst du Regeln an.

DATEI `security/alerts.yaml`

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: security
  namespace: monitoring
spec:
  groups:
    - name: security
      rules:
        - alert: KritischeSchwachstelle
          expr: sum(trivy_image_vulnerabilities{severity="Critical"}) > 0
          for: 1h
          labels:
            severity: warning
          annotations:
            summary: Es laufen Images mit kritischen CVEs.
        - alert: VieleBlockiertePakete
          expr: sum(rate(hubble_drop_total{reason="POLICY_DENIED"}[5m])) > 1
          for: 10m
          labels:
            severity: warning
          annotations:
            summary: Ungewöhnlich viele blockierte Verbindungen.
```

`for: 1h` vermeidet Alarme bei kurzzeitigen Ausreißern. Dank `ruleSelectorNilUsesHelmValues: false` aus Kapitel 10 findet Prometheus die Regel.

MAC Schritt 1

```
kubectl apply -f security/alerts.yaml
```

Lädt die Regeln.

MAC Schritt 2

```
kubectl port-forward -n monitoring \
  svc/kps-kube-prometheus-stack-prometheus 9090
```

Unter http://localhost:9090/alerts siehst du den Zustand der Alarme.

STOLPERFALLE

Der Reason-Wert `POLICY_DENIED` ist ein gängiger Wert von `hubble_drop_total`, kann sich aber zwischen Versionen unterscheiden. Prüfe die echten Werte mit `sum by (reason) (hubble_drop_total)`, bevor du dich auf den Alarm verlässt.

Für Falco legst du einen Grafana-Alarm an: Öffne das Panel **Falco nach Regel**, wähle **More > New alert rule**, setze die Schwelle auf größer als null und speichere. Grafana prüft die LogQL-Abfrage dann regelmäßig.

12.3 Die Angriffsübung

Jetzt spielst du einen Einbruch über alle Schichten durch. Halte dein Dashboard offen. Du übernimmst die Rolle des Angreifers, der eine Remote-Shell im `frontend` erlangt hat.

Bevor du anfängst, hol deine Aufschriebe aus Kapitel 1. Stimmen deine Erwartungen?

Phase 1: Umsehen

MAC Schritt 3

```
kubectl exec -it -n demo deploy/frontend -- sh
```

Das simuliert den Einbruch. **Falco** meldet sofort `Terminal shell in container`.

POD Schritt 4

```
id
```

Du bist Nutzer 100, nicht root. Das ist **PSA**: `restricted` verlangt `runAsNonRoot`.

POD Schritt 5

```
ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Fehler: Das Verzeichnis existiert nicht. Das ist **RBAC** aus Kapitel 3: `automountServiceAccountToken: false`. Ohne Token spricht der Angreifer nicht mit dem API-Server.

Phase 2: Ausbreiten

POD Schritt 6

```
curl -s -m 3 -o /dev/null -w '%{http_code}\n' \
  http://kps-grafana.monitoring.svc.cluster.local/
```

Timeout (`000`). **Cilium** blockiert, weil dieser Weg nicht erlaubt ist.

POD Schritt 7

```
curl -s -m 3 -o /dev/null -w '%{http_code}\n' \
  http://backend:8080/admin
```

Erwartet: `403` . **Cilium** lässt nur `GET /` durch.

Phase 3: Daten ausschleusen

POD Schritt 8

```
curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.org/
```

Timeout (`000`). **Cilium** erlaubt nur `example.com` . Der Weg nach draußen ist zu.

Phase 4: Festsetzen

POD Schritt 9

```
echo test > /usr/bin/hintertuer
```

`Permission denied` . Du bist kein root, das System-Verzeichnis gehört dir nicht.

POD Schritt 10

```
exit
```

Der Angreifer steckt fest.

Phase 5: Der Umweg über den API-Server

Stell dir vor, der Angreifer hat Zugangsdaten von alice gestohlen. Ihr Zertifikat aus Kapitel 3 ist inzwischen abgelaufen, das zeigt schon den Nutzen kurzer Laufzeiten. Für die Übung handelst du deshalb mit `--as` als alice:

MAC Schritt 11

```
kubectl run ausbruch -n demo --image=busybox:1.36 \
  --privileged --as=alice --as-group=plattform
```

alice darf hier Pods erstellen, aber **PSA** lehnt den privilegierten Pod ab.

MAC Schritt 12

```
kubectl get secrets -n kube-system --as=alice --as-group=plattform
```

`Forbidden` . **RBAC** gilt nur in `demo` . Der Versuch steht jetzt im **Audit-Log**.

12.4 Die Spurensuche

Wechsle die Rolle: Du bist jetzt die Verteidigung. Beantworte die Fragen nur aus Grafana.

Frage	Wo du suchst
Wann begann der Vorfall, und in welchem Pod?	Falco in Loki
Welche Verbindungen wurden versucht?	Hubble
Hat jemand auf Secrets zugegriffen?	Audit-Log in Loki
Wer hat einen privilegierten Pod versucht?	Audit-Log, `responseStatus_code="403"`
Was geschah in der API-Anfrage?	Tempo, falls erfasst

MAC

Schritt 13

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  hubble observe --from-pod demo/frontend \
  --verdict DROPPED --last 20
```

Zeigt alle blockierten Versuche des `frontend`. Hubble hält nur die jüngsten Flüsse im Speicher, für einen längeren Zeitraum nimmst du die Metriken.

Diese Abfrage zeigt jede Ablehnung durch alice:

```
{job="audit"} | json
  | user_username="alice"
  | responseStatus_code="403"
```

DAS SOLLTEST DU SEHEN

Du kannst die Geschichte des Angriffs erzählen: Wann er begann, was versucht wurde und wo jede Schicht eingegriffen hat. Genau das ist der Unterschied zwischen einer Sammlung von Tools und einer Architektur.

12.5 Was die Übung gezeigt hat

Aktion	Verhindert von	Erkannt von
Shell im Container	nichts	Falco
Rechte als root	PSA	Audit-Log
Token stehlen	RBAC, kein Mount	nichts nötig
Andere Dienste erreichen	Cilium	Hubble
Ins Internet senden	Cilium	Hubble
Privilegierter Pod	PSA	Audit-Log

Aktion	Verhindert von	Erkannt von
Secrets lesen	RBAC	Audit-Log

Beachte die erste Zeile: Die Shell selbst hat nichts verhindert. Dass der Einbruch nichts bringt, liegt am Zusammenspiel der anderen Schichten.

PROBIER ES SELBST

Suche die Schwächen deiner eigenen Architektur. Was wäre, wenn der Angreifer im `backend` statt im `frontend` gelandet wäre? Was erreicht jemand mit deinem Admin-Zugang auf dem Mac? Welche dieser Lücken schließt keines der Tools? Notiere drei Punkte für deine nächste Lernrunde.

WAS DU GELERNT HAST

- Einzelne Tools werden erst durch Zusammenspiel zu einer Architektur.
- Nicht jede Schicht verhindert, manche erkennen, und beide Rollen sind nötig.
- Eine Übung zeigt, ob deine Annahmen stimmen.
- Ein Dashboard ohne Alarm hilft nur, wenn jemand hinschaut.

KAPITEL 13

Betrieb und Fehlersuche

Du hältst den Stack am Laufen, sicherst das Wichtige und findest Fehler nach einem festen Muster.

Dauer: 30 Minuten Lesen, danach als Nachschlagewerk

13.1 Der Ablauf nach jedem Neustart

Nach einem Neustart der VM startet alles von selbst.

MAC Schritt 1

```
kubectl get nodes
kubectl get pods -A | grep -vE 'Running|Completed'
```

Der zweite Befehl zeigt nur Pods, die nicht in Ordnung sind. Die Liste sollte nach einigen Minuten leer sein.

13.2 Was du sichern musst

Was	Wie	Wo ablegen
k3s-Server-Token	Passwort-Manager	Nicht im Cluster
etcd	Snapshot	Vom Node herunter
Konfiguration	Dieses Repo	Git

NODE Schritt 2

```
sudo k3s etcd-snapshot save --name manuell
```

Erstellt einen Snapshot des Datastores. Das geht, weil du in Kapitel 2 `cluster-init` gewählt hast.

NODE Schritt 3

```
sudo k3s etcd-snapshot ls
```

Zeigt alle Snapshots. k3s erstellt zusätzlich alle zwölf Stunden automatisch einen.

STOLPERFALLE

Ein etcd-Snapshot allein genügt für die Wiederherstellung nicht. Wegen `secrets-encryption` brauchst du auch das Server-Token aus `/var/lib/rancher/k3s/server/token`. Sichere es im Passwort-Manager. Eine Sicherung ist außerdem erst dann eine Sicherung, wenn du die Wiederherstellung einmal geübt hast.

13.3 Das Fehlersuch-Muster

Geh bei jedem Problem von außen nach innen vor. Rate nicht, sondern schau nach.

1. **Was läuft nicht?** `kubectl get pods -A`
2. **Warum?** `kubectl describe pod <name> -n <ns>`, ganz unten stehen die Events.
3. **Was sagt die Anwendung?** `kubectl logs <pod> -n <ns> --previous`
4. **Wer hat es blockiert?** Geh die Tabelle unten durch.

Symptom	Wahrscheinliche Schicht	Prüfen mit
`Forbidden` beim Anlegen	RBAC	`kubectl auth can-i`
`violates PodSecurity`	PSA	Label am Namespace
`admission webhook denied`	Kyverno	`kubectl get polr -A`
Verbindung läuft ins Timeout	Cilium	`hubble observe --verdict DROPPED`
HTTP 403 ohne Anwendungslog	Cilium L7	`hubble observe --verdict DROPPED`
Pod hängt in `ContainerCreating`	Secret oder Volume fehlt	`kubectl describe pod`
Keine Logs in Grafana	Alloy oder Loki	`kubectl logs ds/alloy -n monitoring`
Keine Traces in Grafana	Alloy oder Tempo	`kubectl logs statefulset/tempo -n monitoring`

13.4 Die Notbremsen

Wenn eine Sicherheitsschicht den Betrieb blockiert, kommst du so schnell heraus. Nutze das nur zum Beheben und schalte danach wieder ein.

Schicht	Notbremse
Kyverno	`kubectl delete vpol <name>`, im Extremfall Webhook löschen
Cilium-Policy	`kubectl delete cnp --all -n <ns>`
PSA	Label auf `enforce=privileged` setzen
Audit-Log	Zeilen in `config.yaml` auskommentieren, k3s neu starten

MAC Schritt 4

```
kubectl label namespace demo --overwrite \
  pod-security.kubernetes.io/enforce=privileged
```

Beispiel für PSA. `--overwrite` ist nötig, weil das Label schon existiert.

13.5 Updates

Halte die Reihenfolge ein, denn die Komponenten hängen voneinander ab.

1. **k3s:** immer nur eine Minor-Version auf einmal. Prüfe vorher die Kompatibilität von Cilium.
2. **Cilium:** vorher die Upgrade-Notes lesen.
3. **Kyverno:** In 1.20 fällt `ClusterPolicy` weg. Du nutzt schon `ValidatingPolicy`, bist also vorbereitet.
4. **Alles andere:** Eines nach dem anderen, dazwischen `kubectl get pods -A` prüfen.

Falls Cilium in einer späteren Version Kubernetes 1.37 unterstützt, kannst du auf den Kanal `stable` wechseln. Du änderst dazu nur `INSTALL_K3S_CHANNEL` und führst das Installationsskript erneut aus.

13.6 Ressourcen im Blick

MAC Schritt 5

```
kubectl top node
kubectl top pods -A --sort-by=memory | head -15
```

Zeigt die größten Speicherverbraucher. Vergleiche mit deinem Plan aus Kapitel 1.

Wird der Speicher knapp, geh in dieser Reihenfolge vor:

1. Das Sampling der API-Server-Traces senken oder Tempo abschalten.
2. Prometheus-Retention von zwei Tagen auf einen senken.
3. Trivy nur noch wöchentlich scannen lassen: `scannerReportTTL: 168h`.
4. Einen zweiten Node hinzufügen.

WAS DU GELERNT HAST

- Ein Snapshot ohne Token und ohne Wiederherstellungstest ist keine Sicherung.
- Jede Schicht hat ihre typische Fehlermeldung, und ihre eigene Notbremse.

Wie es weitergehen kann

Du hast jetzt eine vollständige Grundlage. Diese Themen bauen sinnvoll darauf auf:

- **Image-Signaturen:** Cosign und `ImageValidatingPolicy` in Kyverno.

- **GitOps:** Argo CD oder Flux wenden deine Manifeste aus dem Repo an.
- **WireGuard:** Cilium verschlüsselt den Verkehr zwischen Nodes.
- **Mutual TLS:** Cilium authentifiziert auch einzelne Pods gegeneinander.
- **Ein zweiter Node:** Damit lernst du Hochverfügbarkeit für etcd.
- **Der Zusatzteil:** TLS und ein Secrets-Tresor in den nächsten drei Kapiteln.

Zusatzteil

Hier endet der Hauptteil. Die folgenden drei Kapitel sind freiwillig, du brauchst sie für nichts davor.

Sie schließen zwei Lücken, die der Hauptteil offen lässt: Dienste sprechen unverschlüsselt miteinander, und Secrets entstehen durch Skripte, ohne Historie und ohne Protokoll darüber, wer sie gelesen hat.

Kapitel	Inhalt
14	cert-manager: eigene CA und automatische Zertifikate
15	OpenBao: ein Tresor mit Versiegelung, Rechten und Audit-Log
16	External Secrets: Tresor-Werte werden zu Kubernetes-Secrets

Die Kapitel bauen aufeinander auf und setzen Kapitel 10 voraus. Nach einem Neustart der VM musst du OpenBao jedes Mal von Hand entsiegeln, das erklärt Kapitel 15.

KAPITEL 14

Zusatz: cert-manager

Du betreibst deine eigene Zertifizierungsstelle im Cluster und lässt Zertifikate automatisch ausstellen und erneuern.

Dauer: 45 Minuten **Voraussetzung:** Kapitel 10

14.1 Warum Automatisierung

Zertifikate laufen ab, meist nachts und am Wochenende. cert-manager erneuert sie selbstständig. Du beschreibst nur, welches Zertifikat du willst, und es landet als Secret im Namespace.

Objekt	Bedeutung
`Issuer`, `ClusterIssuer`	Wer stellt aus? Im Namespace oder clusterweit
`Certificate`	Was willst du? Namen, Laufzeit, Ziel-Secret
`Secret`	Das Ergebnis: `tls.crt`, `tls.key`, `ca.crt`

Dein Cluster steht im Heimnetz, Let's Encrypt kann ihn also nicht erreichen. Deshalb baust du eine eigene private CA auf. Der Ablauf ist derselbe wie bei einer öffentlichen.

14.2 Installieren

MAC — Schritt 1

```
helm upgrade --install cert-manager \
  oci://quay.io/jetstack/charts/cert-manager \
  --version v1.21.3 -n cert-manager --create-namespace \
  --set crds.enabled=true --wait
```

`crds.enabled=true` installiert die benötigten Objekttypen gleich mit. Ohne das Flag fehlen sie, und `Certificate` wäre unbekannt.

MAC — Schritt 2

```
kubectl get pods -n cert-manager
```

Drei Pods sollten laufen: Controller, CA-Injector und Webhook.

14.3 Die eigene CA aufbauen

Die Kette entsteht in drei Schritten: Ein selbstsignierter Aussteller erstellt das Wurzelzertifikat, und dieses wird dann zur CA, die alles Weitere signiert.

DATEI `security/tls/ca.yaml`

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: selfsigned
spec:
  selfSigned: {}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: k3s2-root-ca
  namespace: cert-manager
spec:
  isCA: true
  commonName: k3s2 Root CA
  secretName: k3s2-root-ca
  duration: 87600h
  privateKey:
    algorithm: ECDSA
    size: 256
  issuerRef:
    name: selfsigned
    kind: ClusterIssuer
    group: cert-manager.io
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: k3s2-ca
spec:
  ca:
    secretName: k3s2-root-ca
```

`isCA: true` macht das Zertifikat zur CA, `duration: 87600h` lässt sie zehn Jahre gelten. Bei einem `ClusterIssuer` sucht cert-manager das Secret im Namespace `cert-manager`, deshalb liegt das Wurzelzertifikat dort.

MAC — Schritt 3

```
kubectl apply -f security/tls/ca.yaml
```

Baut die CA auf.

MAC — Schritt 4

```
kubectl get clusterissuer
kubectl get certificate -n cert-manager
```

Alle Einträge sollten `READY True` zeigen.

WARUM DAS WICHTIG IST

Das Secret `k3s2-root-ca` enthält den privaten Schlüssel deiner CA. Wer ihn hat, kann für jeden Namen in deinem Cluster gültige Zertifikate ausstellen. Das ist ein gutes Beispiel für ein Objekt, dessen Lesezugriff du per RBAC streng begrenzen musst.

14.4 Das erste Zertifikat

DATEI `security/tls/demo-cert.yaml`

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: backend-tls
  namespace: demo
spec:
  secretName: backend-tls
  duration: 2160h
  renewBefore: 720h
  dnsNames:
    - backend
    - backend.demo.svc
    - backend.demo.svc.cluster.local
  issuerRef:
    name: k3s2-ca
    kind: ClusterIssuer
    group: cert-manager.io
```

Das Zertifikat gilt 90 Tage und wird 30 Tage vor Ablauf erneuert. Die `dnsNames` sind alle Namen, unter denen der Service erreichbar ist. Eine Abweichung führt zu einem Zertifikatsfehler beim Client.

MAC — Schritt 5

```
kubectl apply -f security/tls/demo-cert.yaml
kubectl get certificate -n demo
```

Sollte nach wenigen Sekunden `READY True` zeigen.

MAC — Schritt 6

```
kubectl get secret backend-tls -n demo \
  -o jsonpath='{.data.tls\.crt}' | base64 -d \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Liest das Zertifikat aus dem Secret und zeigt Aussteller, Laufzeit und Namen an.

DAS SOLLTEST DU SEHEN

`issuer` lautet `CN=k3s2 Root CA`, `subjectAltName` nennt deine drei DNS-Namen.

14.5 Erneuerung ausprobieren

MAC Schritt 7

```
kubectl get secret backend-tls -n demo \
  -o jsonpath='{.metadata.uid}{"\n"}'
```

Notiere dir die UID.

MAC Schritt 8

```
kubectl delete secret backend-tls -n demo
```

Simuliert einen Verlust.

MAC Schritt 9

```
kubectl get secret backend-tls -n demo \
  -o jsonpath='{.metadata.uid}{"\n"}'
```

Nach kurzer Zeit gibt es wieder ein Secret, mit neuer UID. cert-manager gleicht ständig Soll und Ist ab. Genau das meint das Controller-Prinzip in Kubernetes.

14.6 Grafana per HTTPS erreichbar machen

Bisher nutzt du `port-forward`. Jetzt bringst du Grafana über Traefik ans Netz, mit einem Zertifikat deiner CA.

DATEI `security/tls/grafana-ingress.yaml`

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: grafana
  namespace: monitoring
  annotations:
    cert-manager.io/cluster-issuer: k3s2-ca
spec:
  ingressClassName: traefik
  tls:
    - hosts:
        - grafana.k3s2.local
      secretName: grafana-tls
  rules:
    - host: grafana.k3s2.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: kps-grafana
                port:
                  number: 80
```

Die Annotation genügt: cert-manager sieht den Ingress und legt das passende `Certificate` selbst an.

MAC Schritt 10

```
kubectl apply -f security/tls/grafana-ingress.yaml
kubectl get certificate -n monitoring
```

Das Zertifikat `grafana-tls` entsteht automatisch.

MAC Schritt 11

```
kubectl get secret k3s2-root-ca -n cert-manager \
  -o jsonpath='{.data.ca\.crt}' | base64 -d > k3s2-ca.crt
```

Holt nur den öffentlichen Teil der CA. Diese Datei darfst du weitergeben.

MAC Schritt 12

```
curl --cacert k3s2-ca.crt \
  --resolve grafana.k3s2.local:443:172.16.0.136 \
  https://grafana.k3s2.local/api/health
```

`--resolve` spart den DNS-Eintrag, und `--cacert` vertraut deiner CA.

DAS SOLLTEST DU SEHEN

Die Antwort ist JSON mit `"database": "ok"`. Lässt du `--cacert` weg, verweigert `curl` die Verbindung, weil dein Mac die CA nicht kennt. Genau so soll es sein.

PROBIER ES SELBST

Trage `172.16.0.136 grafana.k3s2.local` in `/etc/hosts` ein und öffne die Adresse im Browser. Welche Warnung erscheint, und was müsstest du in der macOS-Schlüsselbundverwaltung tun, damit sie verschwindet? Lösche die Datei `k3s2-ca.crt` nicht, du brauchst sie noch.

WAS DU GELERNT HAST

- cert-manager gleicht Zertifikate ständig ab und erneuert sie selbst.
- Eine private CA entsteht aus einem selbstsignierten Aussteller und einem CA-Issuer.
- Der Schlüssel der CA ist eines der wertvollsten Secrets im Cluster.
- Eine einzige Annotation am Ingress reicht für automatisches TLS.

KAPITEL 15

Zusatz: OpenBao als Tresor

Du betreibst einen eigenen Secrets-Server mit TLS und Audit-Log und verstehst, wie Versiegeln, Richtlinien und Anmelden funktionieren.

Dauer: 75 Minuten **Voraussetzung:** Kapitel 14

15.1 Warum ein Tresor

Bisher liegen deine Secrets als Kubernetes-Objekte vor, erzeugt von Skripten. Das hat Grenzen: Es gibt keine Historie, keine feinen Rechte pro Pfad und kein Protokoll, wer welchen Wert gelesen hat. OpenBao ist ein quelloffener Fork von Vault und löst genau das.

Begriff	Bedeutung
Seal/Unseal	Der Tresor startet versiegelt, die Daten sind verschlüsselt und unzugänglich.
Unseal Keys	Teilschlüssel, von denen eine Mindestzahl den Tresor öffnet.
Root Token	Allmächtiges Anfangs-Token, nur für die Ersteinrichtung.
Secrets Engine	Speicherort. `kv-v2` ist ein Schlüssel-Wert-Speicher mit Versionen.
Auth Method	Wie sich jemand anmeldet, hier per Kubernetes-ServiceAccount.
Policy	Was ein angemeldetes Token darf, pro Pfad.

WARUM DAS WICHTIG IST

Versiegeln schützt die Daten, wenn jemand die Disk stiehlt oder an den Pod kommt: Ohne Unseal Keys sind die Daten wertlos. Der Preis dafür: Nach jedem Neustart muss jemand den Tresor öffnen. Wer das tut, musst du entscheiden. Bei einem einzelnen Node im Heimnetz bist du das, von Hand.

15.2 Das TLS-Zertifikat vorbereiten

MAC Schritt 1

```
kubectl create namespace openbao
```

Der Standard `baseline` genügt, OpenBao braucht keinen Host-Zugriff.

DATEI `security/tls/openbao-cert.yaml`

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: openbao-tls
  namespace: openbao
spec:
  secretName: openbao-tls
  duration: 2160h
  renewBefore: 720h
  dnsNames:
    - openbao
    - openbao.openbao
    - openbao.openbao.svc
    - openbao.openbao.svc.cluster.local
    - localhost
  ipAddresses:
    - 127.0.0.1
  issuerRef:
    name: k3s2-ca
    kind: ClusterIssuer
    group: cert-manager.io
```

`127.0.0.1` und `localhost` sind nötig, weil der `bao`-Befehl im Pod die lokale Adresse `https://127.0.0.1:8200` anspricht. Das Chart setzt das so.

MAC — Schritt 2

```
kubectl apply -f security/tls/openbao-cert.yaml
kubectl get certificate -n openbao
```

Das Secret `openbao-tls` muss bereitstehen, bevor der Pod startet.

15.3 Installieren

DATEI `openbao/values.yaml`

```yaml
global:
  tlsDisable: false

injector:
  enabled: false

server:
  standalone:
    enabled: true
    config: |
      ui = true

      listener "tcp" {
        address         = "[::]:8200"
        cluster_address = "[::]:8201"
        tls_cert_file   = "/openbao/tls/tls.crt"
        tls_key_file    = "/openbao/tls/tls.key"
      }

      storage "pebbledb" {
        path = "/openbao/data/pebbledb"
```

```
      }

      audit "file" "to-stdout" {
        description = "Audit-Log auf die Standardausgabe"
        options {
          file_path = "stdout"
        }
      }
  dataStorage:
    enabled: true
    size: 1Gi
    storageClass: local-path
  extraEnvironmentVars:
    BAO_CACERT: /openbao/tls/ca.crt
  volumes:
    - name: tls
      secret:
        secretName: openbao-tls
  volumeMounts:
    - name: tls
      mountPath: /openbao/tls
      readOnly: true
  resources:
    requests:
      cpu: 50m
      memory: 128Mi
    limits:
      memory: 256Mi

ui:
  enabled: true
```

`tlsDisable: false` schaltet die Verschlüsselung ein. Der Injector entfällt, weil External Secrets die Werte liefert. Das Audit-Gerät schreibt auf die Standardausgabe, und Alloy bringt es nach Loki. Werte stehen dort nur gehasht im Log. Die Option `log_raw` aus der Doku würde sie im Klartext zeigen, setze sie also nicht.

MAC Schritt 3

```
helm upgrade --install openbao openbao/openbao \
  -n openbao --version 0.30.7 -f openbao/values.yaml
```

Das Repository `openbao` ist bei dir schon eingetragen, sonst `helm repo add openbao https://openbao.github.io/openbao-helm`.

STOLPERFALLE

Verwende hier **kein** `--wait`. Der Pod wird erst `Ready`, wenn der Tresor entsiegelt ist. Helm würde also bis zum Timeout warten und einen Fehler melden, obwohl alles richtig läuft.

MAC Schritt 4

```
kubectl get pods -n openbao
```

`openbao-0` steht auf `Running`, aber `0/1`. Das ist hier der richtige Zustand.

15.4 Initialisieren und entsiegeln

MAC Schritt 5

```
kubectl exec -it openbao-0 -n openbao -- bao operator init \
  -key-shares=3 -key-threshold=2
```

Erzeugt den Hauptschlüssel und zerlegt ihn in drei Teile, von denen zwei reichen. Der Befehl läuft nur einmal im Leben dieses Tresors.

STOLPERFALLE

Die Ausgabe zeigt die drei Unseal Keys und das Root Token nur **dieses eine Mal**. Speichere sie sofort in deinem Passwort-Manager, niemals im Repo und niemals in einer Datei in Dropbox. Verlierst du die Keys, sind alle Daten im Tresor unwiederbringlich verloren.

MAC Schritt 6

```
kubectl exec -it openbao-0 -n openbao -- bao operator unseal
```

Fragt verdeckt nach dem ersten Unseal Key. Schreibe Keys nicht direkt in die Befehlszeile, sonst landen sie in der Shell-Historie.

MAC Schritt 7

```
kubectl exec -it openbao-0 -n openbao -- bao operator unseal
```

Fragt nach dem zweiten Key. Nach dem zweiten von zwei nötigen Teilen ist der Tresor offen.

MAC Schritt 8

```
kubectl exec -it openbao-0 -n openbao -- bao status
```

Zeigt den Zustand.

DAS SOLLTEST DU SEHEN

`Initialized` steht auf `true`, `Sealed` auf `false`. `kubectl get pods -n openbao` zeigt jetzt `1/1`.

15.5 Erste Schritte im Tresor

MAC Schritt 9

```
kubectl exec -it openbao-0 -n openbao -- sh
```

Öffnet eine Shell im Pod. Die folgenden Befehle laufen darin.

POD Schritt 10

```
bao login
```

Fragt verdeckt nach dem Root Token und merkt es sich im Pod für die laufende Sitzung.

POD Schritt 11

```
bao secrets enable -path=secret kv-v2
```

Aktiviert einen versionierten Schlüssel-Wert-Speicher unter dem Pfad `secret/`. Bei einem echten Server ist das nicht von selbst aktiv.

POD Schritt 12

```
bao kv put secret/apps/demo \
  password="erstes-passwort" user=demo
```

Legt zwei Werte unter einem Pfad ab.

POD Schritt 13

```
bao kv get secret/apps/demo
```

Liest sie wieder aus.

POD Schritt 14

```
bao kv put secret/apps/demo \
  password="zweites-passwort" user=demo
```

Schreibt eine neue Version.

POD Schritt 15

```
bao kv get -version=1 secret/apps/demo
```

Holt die alte Version zurück. Das ist die Historie, die einem Kubernetes-Secret fehlt.

15.6 Rechte vergeben: die Policy

Das Root Token darf alles, deine Anwendungen sollen das nicht. In OpenBao ist alles verboten, was nicht erlaubt wurde, ganz wie bei RBAC.

POD Schritt 16

```
bao policy write eso-lesen - <<'EOF'
path "secret/data/apps/*" {
  capabilities = ["read"]
}
path "secret/metadata/apps/*" {
  capabilities = ["read", "list"]
}
EOF
```

Erstellt die Richtlinie `eso-lesen`. Wichtig: Bei `kv-v2` liegen die Werte unter `secret/data/`, nicht unter `secret/`. Das ist die häufigste Fehlerquelle.

Die Richtlinie braucht jetzt noch jemanden, der sie benutzt. Das erledigt Kapitel 16.

POD Schritt 17

```
rm -f ~/.bao-token
exit
```

`bao login` merkt sich das Token in `~/.bao-token`. Ein Root Token gehört nicht in eine Datei, deshalb löschst du sie und verlässt den Pod. Am Ende jeder Sitzung mit dem Root Token gilt: Datei löschen.

15.7 Versiegeln erleben

MAC Schritt 18

```
kubectl delete pod openbao-0 -n openbao
```

Simuliert einen Neustart des Nodes.

MAC Schritt 19

```
kubectl get pods -n openbao
```

Nach kurzer Zeit läuft der neue Pod, bleibt aber auf `0/1`.

MAC Schritt 20

```
kubectl exec -it openbao-0 -n openbao -- bao status
```

`Sealed` steht auf `true`. Die Daten sind noch da, aber unzugänglich, bis du entsiegelst. Dieser Ablauf gehört künftig zu jedem Neustart.

Führe nun zweimal `bao operator unseal` aus, wie oben.

MAC Schritt 21

```
kubectl logs -n openbao openbao-0 --tail=5
```

Hier siehst du das Audit-Log mit JSON-Zeilen. In Grafana findest du es unter `{namespace="openbao"}`.

AUTOMATISCH ENTSIEGELN?

Im Betrieb stört das manuelle Entsiegeln. Die Lösung heißt Auto-Unseal, dabei öffnet ein Cloud-KMS oder ein zweiter Tresor den ersten. Der Schlüssel läge dann aber woanders, und du brauchst diese Infrastruktur. Für dein Lab ist das Handbetrieb die lehrreichere Wahl.

PROBIER ES SELBST

Melde dich an, schreibe eine zweite Policy `nur-lesen-demo`, die **nur** `secret/data/apps/demo` lesen darf, und erzeuge dafür ein Token mit `bao token create -policy=nur-lesen-demo -ttl=1h`. Teste es mit `BAO_TOKEN=... bao kv get secret/apps/demo`. Was passiert bei `secret/apps/anderes`?

WAS DU GELERNT HAST

- Versiegeln schützt ruhende Daten, kostet aber Arbeit bei jedem Neustart.
- Unseal Keys und Root Token gehören in den Passwort-Manager, nie ins Repo.
- `kv-v2` speichert Versionen und legt Werte unter `secret/data/` ab.
- Das Audit-Gerät zeigt, wer was gelesen hat, ohne die Werte preiszugeben.

KAPITEL 16

Zusatz: External Secrets

Werte aus OpenBao erscheinen automatisch als Kubernetes-Secrets, und Änderungen im Tresor kommen von selbst im Cluster an.

Dauer: 60 Minuten **Voraussetzung:** Kapitel 15

16.1 Wie die Teile zusammenspielen

Deine Anwendungen kennen nur normale Kubernetes-Secrets. Sie sollen nichts von OpenBao wissen müssen. Der External Secrets Operator (ESO) schlägt die Brücke:

1. Du beschreibst in einem `ExternalSecret`, welche Werte du brauchst.
2. ESO meldet sich bei OpenBao an, mit einem kurzlebigen ServiceAccount-Token.
3. OpenBao prüft das Token beim Kubernetes-API-Server.
4. ESO holt die Werte und schreibt ein normales Secret.
5. Ändert sich der Wert im Tresor, aktualisiert ESO das Secret.

WARUM DAS WICHTIG IST

Im Git liegt jetzt nur noch die Beschreibung, wo ein Wert liegt, nie der Wert selbst. Damit lässt sich die gesamte Konfiguration gefahrlos einchecken. Das Secret im Cluster bleibt als Kopie bestehen, ist aber dank `secrets-encryption` aus Kapitel 2 im Datastore verschlüsselt.

16.2 Installieren

MAC Schritt 1

```
helm repo add external-secrets https://charts.external-secrets.io
helm repo update
```

Trägt das Repository ein.

MAC Schritt 2

```
helm upgrade --install external-secrets \
  external-secrets/external-secrets \
  -n external-secrets --create-namespace \
  --version 2.14.0 --wait
```

Installiert den Operator mit Webhook und Cert-Controller.

MAC Schritt 3

```
kubectl get pods -n external-secrets
```

Drei Pods sollten laufen.

16.3 Das Konto für den Zugriff

ESO bekommt ein eigenes Konto. OpenBao wird genau diesem Konto vertrauen.

MAC Schritt 4

```
kubectl create serviceaccount eso-openbao -n external-secrets
```

Dieses Konto hat keine Rechte in Kubernetes und braucht auch keine. Es dient nur als Ausweis gegenüber OpenBao.

16.4 OpenBao vertraut Kubernetes

MAC Schritt 5

```
kubectl exec -it openbao-0 -n openbao -- sh
```

Shell im Pod. Der Tresor muss entsiegelt sein.

POD Schritt 6

```
bao login
```

Anmelden mit dem Root Token.

POD Schritt 7

```
bao auth enable kubernetes
```

Aktiviert die Anmeldung per ServiceAccount.

POD Schritt 8

```
bao write auth/kubernetes/config \
  kubernetes_host=https://kubernetes.default.svc:443
```

Sagt OpenBao, wo der API-Server liegt. CA-Zertifikat und Prüf-Token nimmt OpenBao automatisch aus dem eigenen Pod. Das Chart hat seinem Konto dafür die Rolle `system:auth-delegator` gegeben.

POD Schritt 9

```
bao write auth/kubernetes/role/eso \
  bound_service_account_names=eso-openbao \
  bound_service_account_namespaces=external-secrets \
  audience=openbao \
  policies=eso-lesen \
  ttl=1h
```

Verbindet alles: Das Konto `eso-openbao` aus `external-secrets` bekommt die Policy `eso-lesen`. `audience` legt fest, für wen das Token ausgestellt sein muss. So kann ein Token für einen anderen Dienst hier nicht missbraucht werden.

POD Schritt 10

```
rm -f ~/.bao-token
exit
```

Entfernt das gemerkte Root Token und führt zurück zum Mac.

16.5 Die Verbindung beschreiben

DATEI `security/secrets/store.yaml`

```yaml
apiVersion: external-secrets.io/v1
kind: ClusterSecretStore
metadata:
  name: openbao
spec:
  provider:
    vault:
      server: https://openbao.openbao.svc.cluster.local:8200
      path: secret
      version: v2
      caProvider:
        type: Secret
        name: openbao-tls
        namespace: openbao
        key: ca.crt
      auth:
        kubernetes:
          mountPath: kubernetes
          role: eso
          serviceAccountRef:
            name: eso-openbao
            namespace: external-secrets
            audiences:
              - openbao
```

Laut ESO-Doku läuft OpenBao über den `vault`-Provider, weil OpenBao dieselbe API spricht. In den CRDs gibt es inzwischen auch einen eigenen Schlüssel `openBao`, dieses Buch bleibt aber beim dokumentierten Weg. `caProvider` nimmt das Vertrauen aus dem Zertifikat von Kapitel 15. `audiences` muss zur Rolle in OpenBao passen.

MAC Schritt 11

```
kubectl apply -f security/secrets/store.yaml
kubectl get clustersecretstore openbao
```

Die Spalte `STATUS` zeigt `Valid`, `READY` zeigt `True`.

STOLPERFALLE

Bei `InvalidProviderConfig` hilft `kubectl describe clustersecretstore openbao`. Häufige Ursachen: Der Tresor ist versiegelt, die `audience` passt nicht, oder die Rolle nennt ein falsches Konto.

16.6 Das erste externe Secret

DATEI `security/secrets/demo-db.yaml`

```yaml
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: demo-db
  namespace: demo
spec:
  refreshInterval: 1m
  secretStoreRef:
    name: openbao
    kind: ClusterSecretStore
  target:
    name: demo-db
  data:
    - secretKey: password
      remoteRef:
        key: apps/demo
        property: password
```

`refreshInterval: 1m` ist fürs Lernen kurz, im Betrieb reichen eine Stunde oder mehr. `remoteRef.key` ist der Pfad im Tresor, ohne `secret/` und ohne `data/`. `property` wählt den Wert darin. `target.name` ist der Name des entstehenden Secrets.

MAC — Schritt 12

```
kubectl apply -f security/secrets/demo-db.yaml
kubectl get externalsecret -n demo
```

`STATUS` sollte `SecretSynced` zeigen.

MAC — Schritt 13

```
kubectl get secret demo-db -n demo \
  -o jsonpath='{.data.password}' | base64 -d; echo
```

Zeigt den Wert aus dem Tresor. Du hast nie eine Secret-Datei angelegt.

16.7 Ändern und mitverfolgen

MAC — Schritt 14

```
kubectl exec -it openbao-0 -n openbao -- sh
```

Wieder in den Tresor.

POD Schritt 15

```
bao login
```

Anmelden.

POD Schritt 16

```
bao kv put secret/apps/demo \
  password="drittes-passwort" user=demo
```

Ändert den Wert. Nur noch hier wird er gepflegt.

POD Schritt 17

```
rm -f ~/.bao-token
exit
```

Entfernt das Token und führt zurück zum Mac.

MAC Schritt 18

```
kubectl get secret demo-db -n demo \
  -o jsonpath='{.data.password}' | base64 -d; echo
```

Nach spätestens einer Minute erscheint der neue Wert.

STOLPERFALLE

Das Secret ändert sich, die Pods aber nicht zwingend. Ein als **Volume** eingebundenes Secret wird nach kurzer Zeit im Container aktualisiert, die Anwendung muss die Datei allerdings neu lesen. Ein als **Umgebungsvariable** eingebundenes Secret ändert sich dagegen nie im laufenden Prozess. Dann hilft nur `kubectl rollout restart`.

16.8 Wer darf jetzt was?

Prüfe das Modell einmal gegen das, was du gelernt hast:

MAC Schritt 19

```
kubectl auth can-i get secrets -n demo \
  --as=system:serviceaccount:demo:app
```

Erwartet: `no`. Das Konto `app` darf nur Pods lesen. Genau deshalb hast du in Kapitel 3 so eng geschnitten.

Probiere auch den Audit-Blick in Grafana:

```
{namespace="openbao"} |= "secret/data/apps/demo"
```

Damit siehst du jeden Lesezugriff von ESO auf diesen Pfad, mit Zeitpunkt und Konto.

16.9 Lokis Zugangsdaten umziehen

Jetzt übernimmst du eine echte Anwendung. Lokis S3-Secret aus Kapitel 10 wandert in den Tresor.

MAC Schritt 20

```
ACCESS=$(kubectl get secret loki-s3 -n monitoring \
  -o jsonpath='{.data.S3_ACCESS_KEY}' | base64 -d)
SECRET=$(kubectl get secret loki-s3 -n monitoring \
  -o jsonpath='{.data.S3_SECRET_KEY}' | base64 -d)
```

Liest die Werte in Shell-Variablen.

MAC Schritt 21

```
kubectl exec -i openbao-0 -n openbao -- sh -c 'bao login -no-print -'
```

Liest das Token von der Standardeingabe. Füge es ein und drücke `Ctrl-D`. So landet es nicht in der Historie.

MAC Schritt 22

```
printf '{"S3_ACCESS_KEY":"%s","S3_SECRET_KEY":"%s"}' "$ACCESS" "$SECRET" \
  | kubectl exec -i openbao-0 -n openbao -- \
    bao kv put secret/apps/loki-s3 -
```

Überträgt die Werte als JSON über die Standardeingabe. Sie tauchen weder in der Prozessliste noch im Log auf.

MAC Schritt 23

```
unset ACCESS SECRET
kubectl exec openbao-0 -n openbao -- rm -f /home/openbao/.bao-token
```

Entfernt die Werte aus der Shell und das Token aus dem Pod. Prüfe mit `kubectl exec openbao-0 -n openbao -- sh -c 'echo $HOME'`, ob das Home-Verzeichnis bei dir so heißt.

DATEI `security/secrets/loki-s3.yaml`

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: loki-s3
  namespace: monitoring
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: openbao
    kind: ClusterSecretStore
  target:
    name: loki-s3
    creationPolicy: Merge
  dataFrom:
    - extract:
        key: apps/loki-s3
```

`dataFrom.extract` übernimmt alle Schlüssel des Pfades. `creationPolicy: Merge` schreibt in das bestehende Secret, denn das gehört noch dir und nicht ESO.

MAC Schritt 24

```
kubectl apply -f security/secrets/loki-s3.yaml
kubectl get externalsecret -n monitoring
```

Ab jetzt ist OpenBao die Quelle der Wahrheit für dieses Secret.

PROBIER ES SELBST

Migriere nach demselben Muster das Grafana-Passwort. Überlege vorher: Was müsste passieren, damit Grafana ein geändertes Passwort auch wirklich übernimmt?

WAS DU GELERNT HAST

- ESO übersetzt Tresor-Pfade in normale Kubernetes-Secrets.
- Im Git liegt nur noch der Verweis auf einen Wert, nie der Wert selbst.
- OpenBao prüft ServiceAccount-Tokens beim API-Server, und `audience` bindet sie an einen Dienst.
- Umgebungsvariablen ändern sich nie im laufenden Prozess.
- Kubernetes-Secrets sind die Kopie, der Tresor bleibt die Quelle.

Anhang

Versionen

Stand 6. November 2026. Aktualisiere bewusst und einzeln, nicht alles gleichzeitig.

Komponente	Helm-Chart	App-Version
k3s (Kanal `v1.36`)	–	v1.36.6+k3s1
Cilium	1.20.4	1.20.4
kube-prometheus-stack	92.18.0	aus deinem README
Loki (`grafana-community/loki`)	21.0.0	3.7.9
Tempo (`grafana-community/tempo`)	3.2.1	3.1.1
Alloy (`grafana/alloy`)	1.15.0	v1.21.3
Kyverno	3.9.2	v1.19.2
Falco	9.2.2	0.45.2
Trivy Operator	0.38.1	0.36.1
cert-manager (Zusatz)	v1.21.3	v1.21.3
OpenBao (Zusatz)	0.30.7	v2.7.2
External Secrets Operator (Zusatz)	2.14.0	v2.14.0

Dein Repo am Ende

```
k3s/
├── k3s/config.yaml              Node-Konfiguration
├── cilium/
│   ├── values.yaml
│   └── metrics-values.yaml
├── monitoring/
│   ├── kps-k3s-values.yaml
│   ├── loki-values.yaml
│   ├── tempo-values.yaml
│   ├── alloy-values.yaml
│   └── config.alloy
├── seaweedfs/                   Bestehend
├── kyverno/values.yaml
├── falco/values.yaml
├── trivy/values.yaml
├── openbao/values.yaml
└── security/
    ├── audit-policy.yaml
```

```
├── psa.yaml
├── alerts.yaml
├── tracing.yaml
├── rbac/       ServiceAccounts
├── kyverno/    Policies und Tests
├── netpol/     Cilium-Policies
├── demo/       Lab-Anwendung
├── tls/        Zusatz: CA, Zertifikate
└── secrets/    Zusatz: ExternalSecrets
```

In diesem Repo dürfen keine Unseal Keys, Tokens oder privaten Schlüssel liegen. Prüfe das vor jedem Commit.

Spickzettel

Frage	Befehl
Darf X das?	`kubectl auth can-i <verb> <res> --as=<X>`
Was darf X alles?	`kubectl auth can-i --list --as=<X>`
Wen träfe eine PSA-Änderung?	`kubectl label --dry-run=server ...`
Welche Policies gibt es?	`kubectl get vpol`
Welche Verstöße gibt es?	`kubectl get polr -A`
Was wurde blockiert?	`hubble observe --verdict DROPPED`
Welche CVEs gibt es?	`kubectl get vuln -A -o wide`

Für `hubble` und `cilium-dbg` gilt jeweils das Präfix:

```
kubectl -n kube-system exec ds/cilium -c cilium-agent --
```

LogQL-Spickzettel

Ziel	Abfrage
Logs eines Namespace	`{namespace="demo"}`
Text enthalten	`{namespace="demo"} |= "error"`
Text ausschließen	`{namespace="demo"} != "health"`
JSON zerlegen	`{job="audit"} | json`
Feld filtern	`... | json | verb="delete"`
Zählen	`count_over_time({...}[5m])`

Offizielle Dokumentation

- Cilium: https://docs.cilium.io/en/stable/
- k3s Hardening: https://docs.k3s.io/security/hardening-guide
- Kubernetes Pod Security: https://kubernetes.io/docs/concepts/security/pod-security-standards/
- Kubernetes Audit: https://kubernetes.io/docs/tasks/debug-application-cluster/audit/
- Kubernetes Traces: https://kubernetes.io/docs/concepts/cluster-administration/system-traces/
- Kubernetes RBAC: https://kubernetes.io/docs/reference/access-authn-authz/rbac/
- Kyverno: https://kyverno.io/docs/
- Falco: https://falco.org/docs/
- Trivy Operator: https://aquasecurity.github.io/trivy-operator/
- cert-manager: https://cert-manager.io/docs/
- OpenBao: https://openbao.org/docs/
- External Secrets: https://external-secrets.io/
- Loki: https://grafana.com/docs/loki/latest/
- Tempo: https://grafana.com/docs/tempo/latest/
- Alloy: https://grafana.com/docs/alloy/latest/

KUBERNETES ABSICHERN

Ein leerer Ubuntu-Server, ein Laptop und ein Ziel: ein Kubernetes-Cluster, dem du vertrauen kannst.

Dieses Buch baut mit dir Schicht für Schicht eine durchgängige Security-Architektur auf k3s auf. Kein langes Skript zum Abtippen, sondern jeder Befehl einzeln und mit einer Erklärung, warum er nötig ist und woran du erkennst, dass er gewirkt hat.

AUS DEM INHALT

Fundament	k3s mit Cilium statt Flannel und kube-proxy
Zugriff	RBAC und Pod Security Admission
Regeln	Kyverno und Cilium Default-Deny
Laufzeit	Falco erkennt verdächtiges Verhalten
Schwachstellen	Trivy Operator scannt laufend
Nachweis	Audit-Logs, Grafana, Loki und Tempo
Praxis	Eine Angriffsübung gegen das eigene Cluster

Thomas Zachmann
Autor auf Leanpub



Hands-on-Handbuch · Stand: 6. November 2026